Reconfigurable Homomorphic Computing

A Living Textbook

Netlist-Clocked Torus FHE · Differentiable Hardware · Adversarial Polymorphic Ciphers

Digital Defiance / HELUT Project

Living edition 0.1 · epoch 2026-08-13 / C51

Audit stamp: `dirty` · unaudited

For a first university course in a new subject

Canonical source: `textbook/helut-living-textbook.tex`. Corpus of record: `directives/claim-sheet.md`. This book grows when reproducible results grow. Frontier chapters are labeled frontier.

How to read the living boxes

Four boxes appear throughout this book. They are not decoration; they are the difference between a textbook and a press release.

Reproduced (C*n*). A result that someone else can re-run from `REPRODUCE.md` and the claim inventory. Lecture voice may treat it as established *in this corpus*.

Open hedge (H*n*). An honest asterisk. Say it in class with the hedge attached. Do not let students drop the asterisk in a problem set or a paper.

Frontier — not a claim. Speculative territory from `directives/potential-avenues.md`. Assign as a seminar essay, never as a midterm fact.

This section grows when. A stub that is waiting on a new **C** row, a closed **H**, or a graduated avenue. Instructors: skip, or treat as a reading of the matching directive.

Contents

Part I

How This Book Lives

Chapter 1

Preface

This book is the first teaching surface for a subject that did not have a name until the laboratory forced one.

Universities already teach *reconfigurable computing*: LUTs, interconnect, place-and-route, FPGA fabrics, high-level synthesis. They already teach *applied cryptography*: AES, RSA, lattices, sometimes a lecture on fully homomorphic encryption. They almost never teach what happens when the reconfigurable fabric *is* a machine-learning graph API, the bitstream is a Yosys netlist, and the values on the wires are torus samples that must remain encrypted while a host clock ticks flip-flops.

That intersection is the subject of this course. We call the working stack **HELUT**—Homomorphic Edge Look-Up Tensors—and we call the subject **reconfigurable homomorphic computing**.

The stack has three pillars [8, 4]:

1. **Netlist-clocked torus FHE.** Gate-level sequential circuits evaluate as exact $\mathbb{Z}/2^{32}\mathbb{Z}$ tensor graphs, with real bootstrapping-key blind rotation and machine-checkable certificates.

2. **Differentiable hardware cryptanalysis.** Discrete INIT tables melt into continuous tensors; a λ-squeeze grades shatter versus hold; reciprocal structure that combinatoric sieves miss can reappear as an involution.

3. **Adversarial polymorphic ciphers.** Blue ciphers co-evolve under Red pressure and fail closed, rather than waiting to become the next historically leaked stream cipher.

Why a living textbook. Research papers freeze a slice of a laboratory. This course cannot. The claim inventory [3] moves: Metal kernels get faster, hedges close, avenues stay unlabeled until they earn receipts. A professor who taught from a PDF dated June would be teaching a different Metal compiler than a professor teaching from August. So the book is versioned against the corpus *epoch* (2026-08-13 / C51 in this edition) and is honest about stubs.

What this edition is. Edition 0.1 is the *beginnings*: enough foundations to open a course, enough Pillar I to run laboratories, enough Pillars II and III to assign readings, and a frontier part that is labeled as frontier. It is not a complete treatise. Where the laboratory is thin, the page says so.

What this edition is not. It is not a claim that HELUT invented TFHE [1, 11]. It is not a production key-size manual until a lattice estimator fills the pending table (**H1**). It is not a decrypt of U-534 / P1030680. It is not a side-channel paper. Those sentences are printed again in Chapter 3 because students will forget them the week before the final.

Who wrote it. The HELUT project at Digital Defiance. The scientific voice of the book is the same voice as the research-release doctrine: lead with what runs, and with what it does not prove.

How to cite an epoch. Cite the git commit and the claim-sheet epoch, not "the HELUT textbook" as if it were a static ISBN. When a student reproduces **C18**, they are reproducing a dated Metal NTT persist tile, not an eternal constant of nature.

Chapter 2

To the instructor

You are being asked to teach a class that does not yet appear in the ACM/IEEE curriculum guidelines under that name. This chapter is the briefing.

2.1 The course in one paragraph

Students learn to treat a *synthesized gate netlist* as the universal intermediate representation of reconfigurable computing, to lower that netlist onto a *tensor engine* (here: Apple Silicon `MPSGraph`), and to evaluate it either as a boolean-faithful oracle or as torus FHE with certificates. They then learn a second compiler direction—melting discrete hardware into continuous tensors—and a third: designing ciphers that are born under adversarial pressure. The moral of the course is not "FHE is fast on a laptop." The moral is: *reconfigurable computing has a new fabric, and honesty about what that fabric proves is part of the engineering.*

2.2 Prerequisites

Required	Digital logic (LUTs, flip-flops, timing) [13]; linear algebra; a first crypto course (modular arithmetic, attack models) [15].
Helpful	One HDL (Verilog or VHDL); one systems language; Yosys as a black box [18].
Not required	Prior FHE, prior Metal, prior Swift, prior lattice cryptography. The torus chapter is self-contained at the level this course needs.

2.3 Two tracks

Not every department can put an M-series Mac in front of every student.

Systems track. Apple Silicon laboratory. Students build, tick, and certificate-read HELUT from `REPRODUCE.md`. Labs 13 assume this track.

Theory track. No GPU required. Students work the five-cell test, noise lemmas, and published logs as *data*. A department can still grade a claim-audit final (below).

A mixed offering works: theory problem sets for everyone; systems labs as a project option or a second section.

2.4 Suggested grading

Weight	Instrument	Why it exists
30%	Problem sets	Negacyclic arithmetic, packing, certificate reading.
40%	Laboratories or theory equivalents	Receipts, not vibes.
20%	**Claim audit**	Take a public sentence; run the five-cell test from Chapter 3. Fail the assignment if the student upgrades a hedge to a theorem.
10%	Frontier essay	Pick one avenue from Chapter 15. Must label it frontier. Must propose the *first experiment* that would graduate it.

The claim audit is the signature assignment of this course. It trains the skill the field currently lacks: refusing to launder an open hedge into a keynote.

2.5 What to lecture, what to assign as reading

- **Lecture** Parts II–III (foundations and Pillar I), the honesty chapter, and Theorem 10.1 (**C19**).
- **Seminar** Pillar II empirics (shatter / hold, campaign as case study) and Pillar III in edition 0.1. Do not lecture shatter grades as if they were Theorem 10.1.
- **Do not lecture as fact** Chapter 15. Give it as the last two seminars, with the frontier boxes left on the slides.

2.6 How to pull a new edition mid-semester

Science will move under your feet. That is the point of a living textbook.

1. Diff `directives/claim-sheet.md` against the epoch printed on the title page.
2. If a new **C** row appeared, assign the matching lab receipt as a one-page addendum; do not pretend the old PDF contained it.
3. If an **H** closed, tell the class explicitly: "this asterisk is gone; here is the log."
4. If only frontier text moved, do not change the midterm.

Bump `\livingepoch` in `textbook/preamble.tex` when you accept an addendum. Cite git.

2.7 Classroom norms

- Trivial / mock torus encodings are a *shape laboratory*, not FHE. Students who call the boolean oracle "encrypted computing" lose points on purpose.
- Campaign cryptanalysis (Enigma / P1030680) is a *case study* in Pillar II methods, not the FHE claim, and not a decrypt.

- Speculative systems (encrypted LLM firewalls, self-modifying dark binaries, honey-token ledgers) are ethics-and-design discussions, not implementation homework, until they have **C** rows.

This section grows when. A published instructor's guide with slide skeletons, a Moodle/Canvas quiz bank, and a non-Apple reference implementation of the negacyclic oracle for the theory track.

Chapter 3

The living contract

3.1 Why a textbook needs a contract

Ordinary textbooks accrete errata. This one accretes *science*. Without a contract, the lecture slides of 2027 will still be quoting a fused `MPSGraph` megagraph that the laboratory killed in August 2026, or—worse—will quote a speculative avenue as if it were a theorem.

The contract is the research-release doctrine [8] translated into pedagogy.

3.2 The three inventories

Every public sentence in HELUT is classified in `directives/claim-sheet.md`:

Tag	Name	Classroom meaning
Cn	Reproducible result	You may lecture it. Students may cite it. Someone else can re-run it [3].
Hn	Open hedge	You may discuss it. You must keep the asterisk. A problem-set "true/false" that drops the asterisk is false.
Nn	Non-implication	You must actively prevent this inference. It is a common exam trap.

A fourth class lives only in this book and in `directives/potential-avenues.md`:

Frontier	Not a claim until it has a **C** row and a reproduce command. Seminar fuel. Never a midterm fact.

3.3 The five-cell test

For an atomic claim C, the doctrine requires all five, or C is hedged:

1. **Proof** — lemma, reduction, or graded empirical protocol with pass/fail.
2. **Table** — parameters, comparisons, or grades.
3. **Metric** — a number with units and a bar.

4. **Examples** — at least two concrete, runnable instances.

5. **Application** — at least one end-to-end reason to care.

Example 3.1 (Five cells for **C18**)**.** Metal 3-prime NTT persist blind-rotate. Proof: CPU+Metal NTT ≡ schoolbook (19 tests) plus microbench PASS bits. Table: $N \in \{64, 1024\}$ s/BR versus schoolbook persist and fused-EP. Metric: 0.433 s/BR at $N = 1024$ (gpu 0.43 s, RSS 148 MiB). Examples: microbench identity LUT; full_adder boolean SING (**S**timuli **In**, **G**olden out—encrypted evaluation must match the cleartext netlist on the same generic vectors; see Chapter 7). Application: production-shaped polynomial degree on a laptop GPU graph.

Example 3.2 (Five cells for **C19**)**.** TensorLUT continuous→discrete Theorem 1 [10]. Proof: six `TensorLUTFormal.check*` lemmas, aggregated by `certificate()`. Table: lemma vs `holds`; campaign grades kept separate. Metric: $F_{\text{crypto}} = 0$ on unmutated baseline; $\pi = 0$ on binary INIT. Examples: M4 baseline emit; freeze-core involution; blind 3-pair PASS. Application: a stecker search that cannot propose a non-reciprocal map. Asterisk: structural, not a U-534 break (**H6**).

Exercise 3.1 (Claim audit)**.** Take any sentence from a blog post, a grant abstract, or last year's slide deck about "encrypted computing on a Neural Engine." Fill the five cells or mark the sentence a non-claim. If you cannot name the log file, it is not a **C** row.

3.4 Disclosure versus discovery

	Disclosure package	Trajectory
Goal	Someone else re-runs what we assert	We know what to invent next
Artifact	`REPRODUCE.md`, logs, reproduced boxes	trajectory file, Chapter 14
Failure mode	Unreproducible hype	Silence / no next experiment

When a trajectory item graduates, it earns a **C** row (or a tightened **H**) and a reproduce command—*then* it may enter lecture voice. That sentence is the entire editorial policy of this book.

3.5 Standing non-claims

Print these on the course website.

Non-claim. HELUT does not introduce a new lattice assumption, does not claim to have invented TFHE, and does not offer production key sizes until a lattice estimator fills the pending JSON (**H1**). Trivial/oracle Metal graphs are not FHE. `publicMS` gadgets are not a side-channel theorem. Campaign fitness is not encrypted tick rate. TensorLUT involution grades are not a U-534 break. Quantum attacks are out of scope. Metal/GPU power analysis is parked [7].

3.6 How a chapter grows

1. A laboratory result lands in a log under `logs/`.

2. A **C** or **H** row is updated in the claim sheet.

3. The matching chapter gains an example, a table row, or a converted box; `\livingepoch` bumps.

4. The syllabus appendix does *not* automatically change. Instructors opt in (Chapter 2).

This section grows when. Automated extraction of claim-sheet tables into Appendix B, so the snapshot cannot drift. Until then, humans diff the sheet against the appendix before each teaching epoch.

Part II

Foundations of the Subject

Chapter 4

A new subject

4.1 The gap between two mature fields

Reconfigurable computing classically means: a designer writes RTL, a synthesizer emits LUTs and flip-flops, a bitstream configures an FPGA, and the fabric evaluates the circuit at a clock [13, 12]. The fabric is spatially wired. The configuration is a bitstream. The values on the wires are bits.

Fully homomorphic encryption classically means: a scheme evaluates circuits *on ciphertexts* [1, 11]. The "fabric" is an arithmetic library—NTT butterflies, key-switching decompositions, bootstrapping. The configuration is a parameter set. The values on the wires are polynomials in $\mathbb{Z}_q[X]/(X^N + 1)$.

Each field has textbooks. The composition does not.

Definition 4.1 (Reconfigurable homomorphic computing)**.** The study of compiling *already-synthesized* sequential netlists onto *tensor engines* so that LUT evaluation, flip-flop clocking, and (when claimed) torus FHE occupy the same graph object, with explicit certificates of what the graph does and does not prove.

HELUT is one laboratory realization of that definition: Yosys JSON in, `MPSGraph` out, host posedge clock, LWE/GLWE samples on the FHE path [4]. The subject is larger than the prototype. A future chapter may describe a CUDA fabric, an MLIR fabric, or an ASIC LUT plane. The definition does not name Apple.

4.2 Three questions that open the course

1. **Representation.** Can a commodity ML graph API host exact modular arithmetic at $q = 2^{32}$, or does the API silently round?

2. **Clocking.** Can sequential logic—enables, sync resets, a RISC-V core—tick on that graph without the host becoming a bit-banged FPGA?

3. **Honesty.** When the same object grows a real bootstrapping key, what certificates must ship with every tick, and which sentences remain forbidden?

Pillar I is the constructive answer to (1)–(3). Pillar II asks a fourth question the FPGA curriculum never asked: *what if the INIT table is not binary?* Pillar III asks a fifth: *what if the circuit under evaluation is allowed to mutate because the attacker in the next room is the same compiler?*

4.3 Why "reconfigurable" is the right word

An FPGA is reconfigurable because the LUT INIT bits and the interconnect are late-bound. HELUT is reconfigurable in the same sense, with three substitutions:

Classical FPGA	This course	Consequence
LUT INIT bitstream	Yosys `$lut` truth table	Same IR
Routing fabric	Tensor edges in one graph	Place-and-route becomes lowering
Clock tree	Host `graph.run` loop	Timing closure becomes tick protocol
IOB / BRAM	Placeholders and MTLBuffers	Memory is unified, not dual-port
Encrypted?	Optional; certificate-gated	Oracle $\neq$ FHE

The last row is the pedagogical load-bearing wall. Students who have built FPGAs want to say "the CPU is encrypted" the moment PicoRV32 ticks. The boolean oracle path *does* clock PicoRV32 on ciphertext-*shaped* vectors (**C1**). The FHE path *does* fetch NOPs on encrypted PicoRV32 at demo N=8 (**C47**): the PC walks $0, 4, \ldots$, with decrypted Q matching cleartext. Chapter 9 exists so that confusion is a grading event rather than a press cycle.

4.4 The three pillars as a curriculum, not a brand

Roadmap language names them the Turing pillar, the Schneier pillar, and a grand challenge on proving circuits [9]. In this course they are simply I, II, III, in the order a student can verify them:

- Without I, II is a float32 netlist toy and III has nowhere to run.
- Without II, I is a compiler course with FHE homework.
- Without III, I and II only attack the past; they do not design the next cipher under fire.

Edition 0.1 teaches I in lecture depth, Pillar II Theorem 10.1 (**C19**) in lecture depth with empirics in seminar, Pillar III in seminar depth, and the grand-challenge / side-channel items as labeled frontier (Chapter 15).

4.5 Related systems, scoped not ranked

System	Exact q=2^{32} Metal	Netlist clock	BK `$lut` FHE	Diff. hardware
Concrete / tfhe-rs	–	compiler IR	yes	–
OpenFHE / HElib	–	circuit API	yes	–
HELUT boolean oracle	yes	yes	–	–
HELUT encrypted path	yes	yes	yes	–
TensorLUT (Pillar II)	–	Yosys INIT	–	yes

This is a *scope* matrix, not a throughput bake-off [4]. Concrete is a production TFHE compiler; HELUT is a netlist-clocked tensor fabric with an FHE path. Students comparing "ms/gate" across those rows without naming encoding, N, and certificate status fail the five-cell test.

Exercise 4.1. Write four sentences, one per row of the matrix, that a journalist could print without creating a **N**. Then write the one-sentence lie each row most tempts.

This section grows when. A fourth pillar chapter, if FHE-gate / ZK-depth work in the mid-term trajectory [9] graduates. Until then it is Chapter 14, not a `\part`.

Chapter 5

Netlists as the universal IR

5.1 From RTL to LUTs

A combinational Boolean function $f : \{0,1\}^k \to \{0,1\}$ is exactly a truth table of 2^k bits. FPGA vendors package that table as a k-input look-up table. Yosys [18] emits the same object after `abc -lut 2` (or 4, or 6) as a cell of type `$lut` with a parameter string INIT.

Definition 5.1 (Yosys `$lut`)**.** A combinational cell whose output bit is INIT indexed by the concatenated input bits. HELUT treats every `$lut` as one programmable gate, whether the backend is a multilinear oracle or a TFHE blind rotate.

Sequential logic is a `$_DFF*` (or enable / sync-reset variant). The clock is not a net in the tensor graph: it is a *host protocol* (Chapter 7). That substitution—clock as protocol, not as a routed tree—is the first reconfigurable-computing heresy of the course, and it is load-bearing. Tensor engines do not give you a global posedge. They give you `graph.run`.

5.2 Why we start from synthesized netlists

High-level FHE compilers start from a language and lower to homomorphic ops. HELUT starts from a *netlist that already exists*: PicoRV32, a regex DFA, a decision tree, an Enigma core [17]. The pedagogical reason is the same as the systems reason:

- The IR is stable. Verilog front-ends change; `$lut` does not.
- Sequential depth is explicit. Multiplicative depth in an arithmetic circuit is a metaphor; DFF count is a fact.
- Any HDL, any synthesizer that can emit Yosys JSON, is in scope.
- Cryptanalysis targets (historical machines, stream ciphers) already *are* netlists.

Remark 5.1. Starting from netlists does not make HELUT "better than Concrete." It makes it a different compiler problem: *spatial* Boolean fabric onto a graph API, not *scalar* program onto PBS.

5.3 The four objects a student must be able to draw

1. **A LUT** as a box with INIT and k wires.

2. **A DFF** as a box whose Q at tick $t+1$ is a mux of D, enable, and reset, evaluated *between* graph runs.

3. **A batch axis** B as B independent copies of the same fabric in one run—the reconfigurable analog of SIMD, and the reason a Bombe search is a tensor problem.

4. **A polynomial degree** N as the *ciphertext shape*, not the circuit size. $N = 1$ is clear-shape boolean. $N = 1024$ is TFHE-shaped. Confusing N with LUT count is the most common homework error in week 2.

5.4 Worked lowering: a half-adder

A half-adder is two gates: sum $= A \oplus B$, carry $= A \wedge B$. Under additive torus encoding, XOR is a vector add (free). AND is a LUT (or a PBS).

Example 5.1 (Half-adder as a graph)**.** Yosys emits (typically) one `$lut` for the AND and uses an add/xor cell or a second LUT for the sum, depending on techmap. HELUT's Phase-2 compiler [4] lowers this to placeholders for A, B, an add node, a LUT node, and output binds—one persistent `MPSGraph`. There is no FPGA routing step. The "place-and-route" *is* the tensor edge list.

Exercise 5.1. Synthesize a 1-bit full adder with Yosys (`abc -lut 2`) and count `$lut` versus `$_DFF*` cells. Then argue, in four sentences, why the encrypted SING of that netlist is a better first FHE lab than PicoRV32. (PicoRV32 is **C1** on the *oracle* path; the encrypted path's multi-LUT workhorse is the adder, **C6**.)

5.5 Batch is a first-class capacity knob

On an FPGA, duplicating a circuit costs LUTs. On a tensor engine, duplicating a circuit is a batch dimension B on every placeholder. Memory scales as the broadcast intermediates, not as "more CLBs."

Reproduced (C1). Yosys `$lut`/DFF → Metal/CPU tensor eval on the cleartext / trivial-oracle path. PicoRV32, Enigma stream, and campaign-scale batch search all sit on this lowering.

At $N = 1024$, $B = 1000$, Enigma M3's TFHE-shaped working set is gigabytes while clear-shape $N = 1$ stays flat. Students designing a "just crank B" attack are designing a memory bomb. That is a reconfigurable-computing lesson, not an FHE lesson.

Exercise 5.2. Using only the boolean-path scale table in the HELUT README (epoch of this edition), plot tick time versus B at $N = 1$ and $N = 1024$. Identify the regime where graph overhead dominates N, and the regime where $N = 1024$ becomes memory-bound.

5.6 What this chapter refuses to call a bitstream

A Metal graph executable is late-bound, cached, and device-specific. It is tempting to call it a bitstream. We do not, yet. A bitstream has a documented configuration memory; an `MPSGraphExecutable` has an Apple runtime. The analogy is useful in lecture and false on an exam. Chapter 8 returns to this when the compiler stops unrolling schoolbook arithmetic into MLIR and starts calling ring kernels.

Chapter 6

Torus arithmetic and programmable bootstrapping

This chapter is a working minimum of TFHE [1, 11], not a substitute for a lattice-crypto course. The goal is that a student can read a HELUT certificate line without bluffing.

6.1 The torus and the native word

Definition 6.1 (Discrete torus used here)**.** HELUT freezes the modulus $q = 2^{32}$ and represents torus elements as `UInt32`. Addition is native wraparound. This is a *systems* choice: the machine word *is* the torus.

The ring of polynomials is

$$R_q = \mathbb{Z}_q[X]/(X^N + 1),$$

the negacyclic ring. Multiplication by X is a rotation with a sign flip on wrap-around—the algebraic fact that makes blind rotation a barrel shifter on coefficients.

Honesty note 6.1. Exact mod 2^{32} integer tensors are not "homomorphic encryption." They are the arithmetic. Encryption begins when those tensors are LWE or GLWE *samples* with a secret and noise.

6.2 LWE samples, in one page

Definition 6.2 (LWE sample [16])**.** For secret $s \in \mathbb{Z}_q^n$, a sample is $(a, \langle a, s\rangle + e + m) \in \mathbb{Z}_q^n \times \mathbb{Z}_q$, with a uniform and e small Gaussian (or discrete) noise. GLWE is the polynomial analog: mask polynomials plus a body polynomial in R_q.

HELUT's encrypted path uses LWE/GLWE samples and GGSW bootstrap keys. Decision-LWE is bound to IND-CPA by a machine-checkable certificate (**C5**); the bit estimate at production $(n, \sigma) = (1024, 2^{16})$ is calibrated ~ 176 and is *not* a lattice-estimator attack cost until Sage fills the pending JSON (**H1**).

6.3 Programmable bootstrapping, sketched

A TFHE Boolean gate is often evaluated by programmable bootstrapping (PBS): an encrypted phase selects a coefficient of a *test polynomial* via *blind rotation*, then a sample extract returns an

LWE encryption of the LUT output.

Blind rotation is a product of CMUXes, one per bit of the encrypted phase. Each CMUX is an external product against a GGSW encrypt of a secret bit. The accumulator lives in R_q (a GLWE). The sequential dependence across the N (or n) bits is real: you cannot parallelize away the chain inside one bootstrap. You *can* refuse to unroll each external product into $O(N)$ host MLIR nodes. That refusal is Chapter 8.

6.4 Negacyclic Toeplitz embedding

Let $A(X) = \sum_{i=0}^{N-1} a_i X^i$. Negacyclic convolution against a vector x is $M_A x$, where M_A is the Toeplitz matrix whose first column is $(a_0, \ldots, a_{N-1})^\top$ and each subsequent column is the previous column rotated downward with the wrapped entry *negated* [14].

Example 6.1 ($N = 4$ by hand)**.** Let $A = (a_0, a_1, a_2, a_3)$. Then

$$M_A = \begin{pmatrix} a_0 & -a_3 & -a_2 & -a_1 \\ a_1 & a_0 & -a_3 & -a_2 \\ a_2 & a_1 & a_0 & -a_3 \\ a_3 & a_2 & a_1 & a_0 \end{pmatrix}.$$

On `UInt32`, $-a \equiv 2^{32} - a$. The matrix product *is* the ring product. HELUT's Phase-1 kernel proved this bit-exact against a CPU schoolbook oracle, including torus corners $\{0, 1, 2^{31}, 2^{32} - 1\}$.

Exercise 6.1. Expand M_A for $A = (1, 2, 3, 4)$ over $\mathbb{Z}/2^{32}\mathbb{Z}$ and multiply by $x = (1, 0, 0, 0)^\top$. Interpret the output as a polynomial. Then multiply by $x = (0, 1, 0, 0)^\top$ and explain the sign that appears.

Why dense Toeplitz is not the FHE hot path. Materializing $N \times N$ at $N = 1024$ is $\Theta(N^2)$ words per LUT—a kernel proof and a memory stress, not a production PBS. The encrypted hot path moved through tiled schoolbook, fused external product, GPU-resident tiles, and 3-prime NTT (**C16**–**C18**). Students implementing "the HELUT paper kernel" as their FHE engine are implementing 2026-era Phase 1, which the compiler chapter treats as a *control-plane ancestor*.

6.5 Message spacing and Z_{2N}

Blind rotation addresses coefficients at granularity $\delta = q/(2N)$. Packing several LWE bits into one phase must respect the cyclic group Z_{2N}, not an integer that merely looks like a power.

Reproduced (H2 closed). Multi-LUT encrypted full_adder SING PASSes at $N \in \{256, 512, 1024\}$. The bug was arity-3 packing overflowing $256 \cdot 2N$ headroom; the fix is rotation-native encrypt plus `packLWEBits` reduce mod $2N$, with `rotationPower` preferring Z_{2N} representatives.

That closed hedge is the best homework in the chapter: a systems bug that is also an algebraic bug.

Exercise 6.2. Suppose $N = 256$ so $2N = 512$. A pack of three bits each scaled by 256 can leave the $2N$ range. Show a numeric counterexample, then write the reduction that restores a valid rotation power.

6.6 Noise, in the sense this course needs

Three layers ship as code, not as vibes (**C5**):

`TFHENoiseProof`	Discrete ∞-norm inject lemmas.
Gaussian ε-certificate	Ingest failure $\leq 2^{-64}$ under independent-noise hypotheses.
`TFHENoisyBKCertificate`	Depth under a BK-noise bound B_{bk}. Default Metal SING still encrypts BK with $e = 0$. **C22** fills $B_{\mathrm{bk}}/\hat{\sigma}$ from identity-LUT residuals on a covering gadget ($N = 8$: inject $B = 64 \to \hat{\sigma} \approx 6396$; $N = 128$: $\hat{\sigma} \approx 1.47 \times 10^6$, still $< \delta/2$, Gaussian $\varepsilon \log_2 \approx -23.5$ not -64). $\ell = 1$ `booleanPublicMS` cannot carry BK noise (**H4**).

Honesty note 6.2. A noiseless BK is a correct *functional* bootstrap and an incomplete *depth* story. **C22** measures residuals; it does not put noisy BK on the $N = 1024$ Metal SING path. Students who say "we have production noisy BK" from the $N = 8$ table are failing the remaining **H4** asterisk.

6.7 Parameters you may quote

From the cookbook [5], production-shaped boolean:

Knob	Value	Asterisk
N	1024	
q	2^{32}	native word
σ	2^{16}	$\ll \delta/2 = 2^{20}$
Target ε	$\leq 2^{-64}$	Gaussian union on primary wi
Classical target	≥ 128 bits	HELUT est. ≈ 176; **H1**
B_{bk}	0 at $N = 1024$ SING; measured at covering $N = 8$, $N = 128$	**H4**
Inter-LUT refresh	`publicMS` default	lattice-compatible BK masks

Demo $N = 8$ will not meet 128-bit hardness. The certificate reports that honestly. Using demo N for a "secure encrypted CPU" slide is a **N**.

This section grows when. SageMath fills `logs/helut-estimator-results.json` and **H1** can be replaced by an estimator table in this section.

Part III

Pillar I — Netlist-Clocked Torus FHE

Chapter 7

Netlist-clocked torus FHE

Pillar I is the reason the course exists: a synthesized sequential circuit becomes one tensor graph, ticked by the host, with an encrypted backend that is actually LWE.

7.1 The HELUT question

> *Can Apple Silicon's* `MPSGraph` *host exact torus modular arithmetic and gate-level sequential circuits—and can that same hardware object carry real bootstrapping keys without lying about it?*

The first half is a systems existence proof. The second half is a certificate problem. HELUT answers both, with the oracle path and the FHE path named as different objects [4, 8].

7.2 Pipeline

$$\text{Verilog} \xrightarrow{\text{Yosys}} \text{JSON netlist} \xrightarrow{\text{HELUTCore}} \text{one } \texttt{MPSGraph}$$

Host Swift loop = posedge clock; DFF Q ping-pongs across ticks

`$lut` backend = oracle (multilinear / trivial PBS) or encrypted (BK blind rotate)

All FHE-path arithmetic tensors remain `MPSDataType.uInt32`. Float32 belongs to TensorLUT (Pillar II), not to this chapter.

7.3 Compiler lowering

`YosysGraphCompiler` walks a `write_json` module:

1. **Inputs.** Each input port bit becomes an `InputNode` placeholder of shape $[B, N]$.
2. **State.** Each recognized flip-flop allocates a Q placeholder *before* LUT lowering so sequential feedback nets resolve.
3. **LUTs.** Cells schedule until all drivers exist; combinational cycles abort.
4. **Next-state.** Enables and sync-resets become exact $\mathbb{Z}/2^{32}\mathbb{Z}$ muxes, e.g. active-high enable

$$Q_{\text{next}} = E \cdot D + (1 - E) \cdot Q.$$

5. **Outputs.** Driven nets bind to port tensors; Yosys `"x"` becomes constant 0.

Cell coverage includes `$_DFF*`, `$_DFFE*`, `$_SDFF*`, `$_SDFFE*`, `$_SDFFCE*`—the techmap PicoRV32 actually emits.

7.4 Host clock

Each tick is one `graph.run`. With `retainHistory=false`, two preallocated state buffer sets ping-pong and output scratch is reused, so a 1 000-tick stress test does not leak `MTLBuffer` retention. Primary port feeds (including scripted `resetn`) are host-mutable shared buffers; LUT matrices or BK packs are uploaded once and cached.

This is the reconfigurable clock tree: not routed silicon, but a protocol with explicit buffer ownership.

7.5 SING as the teaching metric

SING expands to **S**timuli **In**, **G**olden out.

In hardware verification, a *golden model* is the flawless reference circuit: feed test vectors (stimuli) in, expect the exact right bits out. For HELUT the golden model is the cleartext Yosys netlist simulator. A SING check (`-sing` / `-bench-encrypted-metrics`) is the primary teaching metric and the laboratory analogue of a scan chain: encrypt the same generic stimuli, evaluate under GGSW blind-rotate, decrypt, and demand bit-exact agreement with cleartext.

A passing encrypted tick under SING is not only a boolean match. The log is graded as a cryptographic bundle:

- **Functional:** ciphertext decrypts to the clear netlist (the hard gate).
- **Discrete:** ∞-norm inject lemmas hold where the path prints them.
- **Gaussian:** ingest failure probabilities stay under stated hypotheses.
- **Hardness:** calibrated classical bit estimates print with **H1** attached until the estimator row is honest.
- **Depth:** noisy bootstrap-key residuals are modeled or measured; default Track A throughput SING may still use e=0 BK (**H4**); covering noisy SING receipts are separate claims (**C28**, **C33**–**C37**, **C41**, **C43**).

Brutal engineering rule: if SING fails, you do not get to discuss milliseconds. Throughput and timing benches are irrelevant until the FHE path certifies its own correctness against the golden netlist.

Laboratory 7.1 (Encrypted adder SING)**.** Run the cookbook SING at demo N and at production-shaped N (Appendix D). Record pass/fail, ms/row, and which certificate types printed. Then answer: which of those numbers would you be willing to put in an abstract without an asterisk?

7.6 Two backends, one netlist

Backend	Claim status	Use in this course
Trivial / multilinear / CMUX oracle	Shape laboratory, *not* FHE	Batch scaling, PicoRV32 boot, Enigma letter clocks.
`encrypted` (LWE + GGSW BK)	Pillar I FHE claim (**C4**–**C6**)	SING vs cleartext; certificates on every tick.

Inter-LUT refresh defaults to `publicMS` (lattice-compatible BK masks). `.secret` remains for coverage. `.none` is a debug foot-gun.

Reproduced (C4–C6). Encrypted LWE/GLWE + GGSW blind-rotate per `$lut`; encrypted ≡ clear multi-netlist SING; full_adder through $N = 1024$ (publicMS and secret).

Reproduced (C38). Encrypted host-clocked DFF Q on a 4-bit counter (6 `$lut` + 4 `$_DFFE_PP_`): CPU N=8 and Metal N=1024 public-ms boolean SING PASS. Not PicoRV32; fused metal-netlist and cpu-ggsw remain combinational-only.

Reproduced (C40). Encrypted toy ISA: 4-bit accumulator, `NOP` or `ADD imm`, 11 `$lut` + 4 DFFs. Metal N=1024 SING PASS.

Reproduced (C45). Encrypted PicoRV32: 4785 `$lut` + 1565 DFFs. CPU public-ms boolean output SING PASS at demo N=8, one host posedge, 76.12 ms/row, hardness 4.0 bits.

Reproduced (C46). Encrypted PicoRV32 10-tick `resetn` boot (hold 3). PASS 0.972 s/10 (97.20 ms/row) with $Q \equiv$ clear. Plain `$_DFF_P_` must copy native D (no second public-MS). N=32 1-tick also PASS (819 ms/row, 16-bit hardness); N=64 1-tick PASS (4.35 s/row, 32-bit hardness).

Reproduced (C47). Encrypted PicoRV32 NOP ROM: always-ready `addi x0,x0,0`. Ten instruction fetches, `mem_addr` $0, 4, \ldots,$ `0x24`, $Q \equiv$ clear. 3.15 s/32 at demo N=8. Receipt: `logs/helut-encrypted-n8-cpu-sing-picorv32-nop-fetch.log`.

Reproduced (C49). Encrypted PicoRV32 tiny program: `addi x1,x0,1`, NOP, `sw x1,0(x0)`. Store `wdata`=1 into host RAM. PASS 3.79 s/48 (78.90 ms/row) at demo N=8. `$_SDFFCE_*` enable gates reset; `rdata` held for NBA `mem_xfer`; host DFF copies native LWE.

Reproduced (C50). Same ROM plus `lw x2,0(x0)`. Host RAM on the NBA `mem_xfer` cycle: `rdata`=1. PASS 3.77 s/48 (78.53 ms/row). First-cycle LOAD log may still show leftover NOP 0x13; PicoRV samples the xfer. Receipt: `logs/helut-encrypted-n8-cpu-sing-picorv32-prog-lw.log`.

Reproduced (C51). Metal PicoRV32 NOP-fetch at demo N=8: tiled-kernel BR (`-metal-br-tile 8`; fused default traps). Two fetches (0, 4). PASS 63.55 s/8 (7.94 s/row). Per-LUT Metal BR, not fused metal-netlist.

7.7 Why PicoRV32 is real in FHE, with a speed qualifier

PicoRV32 lowers to 4 785 LUTs and 1 565 DFFs [17]. On the boolean-oracle path it compiles in ∼1.3 s and ticks at ∼173 ms ($N = 1024$, $B = 1$) in the 2026-08-12 boolean bench (**C1**). That is a fabric existence proof: a commodity neural/GPU graph can host a soft CPU's *shape*.

The encrypted path uses the same host-clocked sequential seam as the toy counter (**C38**) and toy ISA (**C40**). Claim **C45** is one encrypted host posedge of the full PicoRV32 netlist at demo N=8 (output SING). Claim **C46** is the PRD 10-tick scripted `resetn` boot at the same N: CPU public-ms boolean SING PASS in 0.972 s (97.20 ms/row), calibrated hardness 4.0 bits, with decrypted Q matching the clear register file. A second public-MS on already-native D for plain `$_DFF_P_` cells had been silently folding $1 \to 0$ — outputs could still match. Claim **C49** is execute+store: `addi x1,x0,1` then `sw`, `wdata`=1, 3.79 s/48 (78.90 ms/row). Claim **C50** is load-back: `lw` sees stored 1 on the NBA xfer, 3.77 s/48 (78.53 ms/row). Claim **C51** is the first Metal sequential PicoRV receipt: NOP-fetch, two instruction addresses, tiled BR, 7.94 s/row at demo N. Metal N=1024 is not SING-logged this epoch — the LUT-tax is the remaining engineering, not a missing compiler.

Honesty note 7.1. "We executed encrypted RISC-V" means **C50** on CPU (addi/store/load of 1) and **C51** on Metal only as a NOP-fetch at demo N (~ 8 s/row). Not Linux; not Metal N=1024. Avenue 2 in Chapter 15 is self-modifying dark code, not this receipt.

7.8 Threat / fidelity split

In scope for the *datapath*	Exact modular eval; shape $[B, N]$; DFF routing; Yosys coverage for PicoRV32.
In scope for the *FHE claim*	LWE/GLWE + BK; noise / ε / hardness / noisy-BK *certificates* as implemented.
Out of scope unless measured	Side channels; quantum; estimator-backed production keys (**H1**); noisy BK on the $N = 1024$ Metal SING path (**H4**).

The split is intentional. If the tensor datapath cannot boot a CPU under mock encodings, adding real TFHE noise only makes the systems problem harder. If the FHE path cannot SING an adder, a CPU slide is theatre.

This section grows when. Metal N=1024 PicoRV tick; fused sequential metal-netlist.

Chapter 8

The Metal torus compiler

8.1 The picture that changed the compiler

```
Yosys $lut  -->  pack LWE  -->  blind rotate (N CMUXes)
                                 |
                                 +-- failure: each external product expands
                                 |    sum_j b_j * X^j a  as O(N) MPSGraph ops
                                 |    --> millions of host MLIR nodes
                                 |    --> encode wall >> run wall
                                 |
                                 +-- design: small graph that *calls* ring kernels
                                     encode cheap; run on Metal; cache
```

CMUX dependence across bootstrap bits is sequential and real. The pain was never "CMUX cannot run on a GPU." The pain was *encoding a schoolbook expansion that should never have been a graph* [6].

Reproduced (fused $N = 1024$ DNF). Schoolbook-in-`MPSGraph` at $N = 1024$ spent 11.6 h on host encode, never reached the GPU, and died `SIGTERM 143`. That log is part of the curriculum: a compiler that does not finish is a measurement, not an anecdote.

8.2 Doctrine

SoftBus / ANE remains a graph machine. We stop mistaking "unroll all ring math into MLIR" for "use the GPU." `MPSGraph` is a *staged IR* and a *scheduler*. Polynomial multiplication and the external product are first-class GPU operations.

8.3 Phase 1 — survive $N = 1024$

Phase 1 does not change the cryptography. It changes the control plane.

Lever	Idea	Teaching point
Tiled blind rotate	CMUX windows of width W; re-ingest ACC	Encode cost scales with W, not fused $N\times$ schoolbook.
Constant CSE	Shared splat banks per tile	Graph builders die on `constantWithScalar` spam.
Executable cache	Compile once per $(N, \ell, W, \mathrm{BK})$	Trial 1 is cold; ticks ≥ 2 must hit cache.
Telemetry	`encode` / `gpu_run` / `host_repack`	Silence is not a hang if the log counts tiles.

Phase 1 without Phase 2 still leaves $O(W \cdot N)$ encode inside each tile. Phase 2 without Phase 1 is a kernel drop into an unmeasured fused path. The laboratory did both, in that order.

8.4 Phase 2 — ring kernels

1. **Negacyclic poly-mul** as a Metal kernel, bit-identical to CPU schoolbook (**C18** NTT $\equiv$ schoolbook, 19 tests).
2. **Fused external product** (**C16**): one `helut_ggsw_external_product` launch per CMUX. $N = 1024$ microbench 1.043 s/BR.
3. **GPU-resident BR tile** (**C17**): ACC + BK on device; 0.519 s/BR at $N = 1024$; $N = 64$ at 0.001 s/BR.
4. **3-prime NTT persist tile** (**C18**): gadget digits in-tile; BK in NTT domain; pointwise EP + iNTT + CRT. 0.433 s/BR at $N = 1024$ (gpu 0.43 s, RSS 148 MiB).

Default Metal BR: fused if $N \leq 64$, tiled-kernel otherwise (NTT EP inside tiles). Legacy fused megagraph is `-metal-br-fused` only, and at production N it is a museum exhibit.

8.5 Numbers a student may quote (epoch 2026-08-13 / C51)

Path at $N = 1024$	s/BR (micro)	SING / 8 rows	RSS
Fused schoolbook megagraph	DNF (11.6 h)	—	∼4 GiB
Poly-mul kernel	3.645	—	—
Fused EP (**C16**)	1.043	25.1 s (3.14 s/row)	—
Persist schoolbook (**C17**)	0.519	12.2 s (1.52 s/row)	68 MiB
NTT persist (**C18**) serial SING	**0.433**	15.6 s (1.95 s/row)	148 MiB
Wavefront-parallel (**C20**)	0.420 (fused 3-prime)	10.6 s (1.33 s/row)	—
Crypto $\ell = 2$ SING (**C21**)	—	11.38 s (1.42 s/row)	—
CPU SING full_adder	—	∼52 s	—

Reproduced (C20, C21). Wavefront-parallel independent `$lut` BRs: boolean SING 10.6 s/8 beats persist-schoolbook **C17**; cryptoPublicMS $\ell = 2$ SING 11.38 s/8 vs **C14**'s 175.6 s.

Open hedge (H3). The scheduler, not a faster NTT, is what beat **C17** on SING. Serial in-tile NTT (**C18**) still loses boolean SING (15.6 s). Sage **H1** is independent of these wall-clocks.

Exercise 8.1 (Compiler post-mortem)**.** In 400 words: why did the fused megagraph never reach the GPU? Your answer must distinguish encode wall from run wall, and must not claim that "Metal cannot multiply polynomials."

Exercise 8.2 (Reading **H3**)**.** Is **C18** a success or a failure? Defend a one-sentence answer that a claim-audit grader will accept. (Hint: both, along different bars.)

8.6 Punchline, for the lecture slide

Phase 1 makes encoding *finish*. Phase 2 makes encoding *irrelevant*. Together they are a SoftBus-native torus compiler: staged IR + ring kernels + cached executables—instead of a schoolbook megagraph hoping the GPU will eventually appear.

This section grows when. NTT EP inside the crypto $\ell = 2$ tile (C21's SING log still shows $W = 64$ schoolbook tiles); then **H3** is only Sage **H1**.

Chapter 9

Certificates as first-class artifacts

Most compiler courses grade a binary that runs. This course grades a binary that *refuses* when it cannot certify.

9.1 The surface

Certificate	Role
`TFHENoiseProof`	Discrete ∞-norm inject lemmas.
`TFHEAsymptoticSecurityCertificate`	Gaussian ingest $\varepsilon \leq 2^{-64}$.
`TFHELWEHardnessCertificate`	Decision-LWE $\rightarrow$ IND-CPA + bit estimate.
`TFHELWECalibration`	Anchor table for the estimator.
`TFHENoisyBKCertificate` / `TFHENoisyBKMeasurement`	Depth under B_{bk}; identity residual $\hat{\sigma}$ (**C22**).
`TFHELWEEstimatorProtocol`	External lattice-estimator merge.

`TFHENoise.refuse` if no certificate. That is not an API flourish. It is the difference between a demo and a claim.

9.2 How to read a SING log

SING = **S**timuli **In**, **G**olden out (Section 7.5): encrypted evaluation must match the cleartext netlist on the same generic vectors. A passing encrypted tick is then read as a bundle:

- Functional: ciphertext decrypts to the clear netlist (the SING gate).
- Discrete: ∞-norm lemmas held on the inject.
- Gaussian: ingest ε under the stated hypotheses.
- Hardness: calibrated classical bits, with **H1** attached until Sage fills the estimator JSON.
- Depth: noisy-BK model; **C22** measured at covering gadget; **C26** graded fail at product N=1024 inject on `cryptoPublicMS`; covering noisy Metal SING exists through inject B=32 (**C36**); torus-scale $B\sim$128 and default Track A e=0 BK remain **H4** asterisks.

Students who report only milliseconds have not completed the lab.

Reproduced (C22). Identity-LUT residual under a covering gadget fills B_{bk} and $\hat{\sigma}$. At $N = 8$, inject $B = 64$ gives $\hat{\sigma} \approx 6396 \ll \delta/2$. At $N = 128$, $\hat{\sigma} \approx 1.47 \times 10^6$, still ∞-norm decodable, Gaussian union $\approx 2^{-23.5}$ not 2^{-64}.

Reproduced (C26). Product-shaped N=1024 identity residual: inject B=64 undecodable on `cryptoPublicMS`/`crypto`; inject B=4 on `.crypto` may be ∞-norm OK with $\varepsilon \log_2 \approx -1$ (not -64). Receipt: `logs/helut-noisy-bk-measure-n1024.log`.

Reproduced (C28). Track B: Metal `cryptoPublicMS` full_adder SING at covering N=128 with inject B=64 PASSes ($B_{\text{bk}} \approx 1.62 \times 10^6$ decodable). Receipt: `logs/helut-encrypted-n128-metal-sing-crypto-noisy.log`.

Reproduced (C29). Theorem $3'$: for any power-of-two torus word w, exact public-MS covering among practical degrees stays $\{8, 128\}$. Widening the limb does not unlock N=1024 (baseLog = 11 never divides 2^k).

Reproduced (C30). At covering N=128, inject $B \leq 8$ prints $\varepsilon \log_2 = -\infty$, but $\hat{\sigma}$ remains $\sim 10^4$–10^5. At B=64, $\varepsilon \log_2 \approx -24$ (**C22**). Receipt: `logs/helut-noisy-bk-eps-sweep-n128.log`.

Reproduced (C31). Incomplete public-MS gap: N=1024 uncoveredBits= 10. Approx candidate is covering `.crypto` (baseLog= 8, $g_0 \neq \delta$). Statement: `directives/ggsw-incomplete-covering.md`.

Reproduced (C32). At N=1024, `.crypto` inject B=1 is ∞-norm decodable; $\varepsilon \log_2 \approx -8.4$ at 8 trials (not -64); `cryptoPublicMS` remains undecodable for $B \geq 1$. Receipts: `logs/helut-noisy-bk-measure-n1024-B1.log`, `logs/helut-noisy-bk-measure-n1024-B1-t8.log`.

Reproduced (C33). Track A Metal SING with covering `.crypto` at N=1024, inject B=1: secret-refresh and public-ms covering paths both PASS (non-zero decodable B_{bk}). Receipts: `logs/helut-encrypted-n1024-metal-sing-covering-crypto-noisy-B1.log`, `logs/helut-encrypted-n1024-metal-sing-covering-publicms-noisy-B1.log`.

Reproduced (C34). Finer covering baseLog= 4 at N=1024: inject B=1 yields $\hat{\sigma} \approx 2.95 \times 10^4$, asymptotic $\varepsilon \log_2 \approx -913$ (meets $\leq 2^{-64}$); Metal secret + public-ms covering-b4 both PASS. Receipts: `logs/helut-noisy-bk-covering-b4-n1024-B1.log`, `logs/helut-encrypted-n1024-metal-sing-covering-b4-noisy-B1.log`. Torus-scale B=128 on the same gadget is still undecodable.

Reproduced (C35). Covering baseLog= 2 at N=1024: $\varepsilon \le 2^{-64}$ holds through inject B=16 ($\varepsilon \log_2 \approx -65.4$; B=32 drops to ≈ -40.5). Metal secret + public-ms covering-b2 at B=16 both PASS ($B_{\mathrm{bk}} \approx 1.86 \times 10^5$ / 6.15×10^4). Receipts: `logs/helut-noisy-bk-covering-b2-eps-vs-B.log`, `logs/helut-encrypted-n1024-metal-sing-covering-b2-noisy-B16.log`. Torus-scale B∼128 remains $\varepsilon \approx -1.6$.

Reproduced (C36). Covering baseLog= 1 at N=1024: $\varepsilon \le 2^{-64}$ holds through inject B=32 ($\varepsilon \log_2 \approx -139$ at 4 trials; B=64 ≈ -26). Metal secret + public-ms covering-b1 at B=32 both PASS. Receipts: `logs/helut-noisy-bk-covering-b1-n1024-B32.log`, `logs/helut-encrypted-n1024-metal-sing-covering-b1-noisy-B32.log`. Finest covering does *not* unlock B=128 ($\varepsilon \approx -0.6$).

9.3 Calibration is not estimation

Label	n	σ	HELUT	Estimator	$\lvert\Delta\rvert$
demo-N8	8	2^{12}	4.0	33.1	29.1
weak-n256	256	2^{17}	40.4	53.7	13.3
mid-n512	512	2^{16}	95.5	92.4	3.2
classic-n630	630	2^{15}	129.0	106.5	22.5
n768-s16	768	2^{16}	135.6	135.4	0.2
prod-n1024-s16	1024	2^{16}	175.7	**180.2**	4.5
n1024-s17	1024	2^{17}	160.7	189.9	29.3
n2048-s16	2048	2^{16}	336.1	369.8	33.7

$q = 2^{32}$, binary secret, native Sage 10.9 + lattice-estimator (**C23**). Merge tolerance is 16 bits: four rows pass, four fail.

Open hedge (H1). Quote "∼ 176 calibrated classical bits at production (n, σ)" together with "estimator 180.2 on prod-n1024-s16." A midterm bubble that says "176-bit secure" without naming the estimator row, or that treats every calibration anchor as estimator cost, is wrong.

9.4 Five-cell test, applied to certificates

Certificates are themselves claims. They need the five cells:

1. Proof: the lemma in code (`TFHENoiseProof`, ...).
2. Table: the calibration table; the cookbook production row.
3. Metric: $\varepsilon \le 2^{-64}$; bit estimates; B_{bk}.
4. Examples: demo $N = 8$ (honest fail of 128-bit) and production $N = 1024$.
5. Application: refuse-closed encrypt so a netlist tick cannot silently drop hardness.

Exercise 9.1. C22 filled the measurement cell at covering-gadget $N = 8$ and $N = 128$. Which cell is still missing for noisy BK as a *product* parameter at $N = 1024$ Metal SING? Name the gadget obstruction ($\ell = 1$ vs covering).

9.5 Honesty as an engineering interface

The non-claims of Chapter 3 are not ethics theater. They are API documentation for speech:

- Oracle Metal graphs expose the *shape* of encrypted evaluation (batch, N, clock). They do not expose Decision-LWE.
- `publicMS` is an on-lattice refresh intent. It is not a leakage proof (**N** on side channels).
- A campaign's settings/second is a cleartext Metal number. It does not certify encrypted tick rate.

Laboratory 9.1 (Certificate literacy)**.** Run `.build/release/helut -hardness-table` and classify each printed row as demo, weak, or production-shaped. For each row, write one sentence a journalist may print and one sentence they may not.

This section grows when. The four off-tolerance anchors (demo-N8, classic-n630, n1024-s17, n2048-s16) get a documented estimator model, or HELUT's core-SVP fit is revised; then **H1** can shrink further.

Part IV

Pillar II — Differentiable Hardware

Chapter 10

Continuous hardware, discrete silicon

The structural theorem of this chapter is lecture-safe (**C19**). The separable melt–freeze–snap certificate is lecture-safe (**C44**). Shatter / hold grades and campaign empirics remain seminar-depth and are *not* the theorem [10].

This section grows when. A completeness theorem for *multi-LUT topological* melt: that melt recovers cryptographic structure combinatoric sieves miss, for a stated class of netlists beyond a fully observed 1-LUT. Theorem 1″ (**C44**) is the separable interpolant, not that class. Stream-cipher melts remain on the trajectory.

10.1 The problem boolean search cannot see

A combinatoric sieve on a cipher netlist proposes discrete genotypes: INIT bits, plugboard pairs, rotor orders. On a short ciphertext the search space is both enormous and full of linguistic hallucinations (the P1030680 campaign is the case study, not the claim). Boolean search *rejects*. It does not *propose* a nearby reciprocal structure that is almost a stecker.

Pillar II melts the hardware [4, 10].

10.2 Setup

Let L be the number of LUT6 cells and $w \in [0,1]^{64L}$ the concatenated INIT genome. Let $y(w)$ be the *multilinear extension* of those INITs on $[0,1]$ (exact on $\{0,1\}$ inputs). Let t be a target bit vector of matching width. Soft crypto fitness and discreteness penalty:

$$F_{\text{crypto}}(w) = -\|y(w) - t\|_2^2, \tag{10.1}$$

$$\pi(w) = \sum_i w_i(1 - w_i), \tag{10.2}$$

$$F(w) = F_{\text{crypto}}(w) - \lambda\,\pi(w), \qquad \lambda \geq 0. \tag{10.3}$$

Remark 10.1 (Implementation schedule). HELUT's GA uses a squeeze $\lambda = \lambda_{\max}(p')^2$. Theorem 1 needs only $\lambda \geq 0$.

Emitter: $E(w)_i = \mathbf{1}[w_i \geq \frac{1}{2}]$. Freeze mask $M \subseteq \{1, \ldots, 64L\}$ removes frozen coordinates from π. Stecker genotypes are *partial involutions* (disjoint pairs) on $\{0, \ldots, 25\}$.

Every Yosys `$lut` of width $1 \ldots 6$ pads to a 64-wide `Float32` INIT. The forward pass is branchless and GPU-shaped. It is not torus FHE.

10.3 Theorem 1 (structural)

Reproduced (C19). TensorLUT continuous→discrete Theorem 1: six machine-checked lemmas (`TensorLUTFormal.certificate()`). Not a cryptanalytic break. Statement: `directives/tensorlut-theorem.md`. Reproduce: `swift test -c release -filter testTensorLUTFormalCertificate`.

Theorem 10.1 (TensorLUT continuous→discrete)**.** *Assume the hypotheses on the certificate: the LUT is multilinear; the GA mutates only unfrozen $w_i \in [0,1]$; the involution sandwich freezes core INITs. Then:*

1. ***Discreteness.*** *$\pi(w) \geq 0$, with equality iff $w \in \{0,1\}^{64L}$.*
2. ***Crypto MSE.*** *$F_{\text{crypto}}(w) \leq 0$, with equality iff $y(w) = t$.*
3. ***Combined objective.*** *$F(w) \leq F_{\text{crypto}}(w)$; if $\pi(w) > 0$, increasing λ strictly decreases F.*
4. ***Emitter.*** *E is idempotent on $\{0,1\}^{64L}$ and agrees with threshold $\frac{1}{2}$.*
5. ***Involution sandwich.*** *Overlapping pairs are rejected; applying a valid pair-set twice is the identity on the alphabet.*
6. ***Freeze.*** *Coordinates in M do not contribute to π.*

Proof (machine-checked). Each clause is a `TensorLUTFormal.check*` in `TensorLUTFormal.swift`, aggregated by `TensorLUTFormal.certificate()`.

Reproduced (C25). Theorem 1 corollary: emitter–discrete agreement ($\pi = 0 \Rightarrow E(w)$ recovers INIT bits) and involution under freeze (`mutatedPreserving` retains frozen pairs). `TensorLUTFormal.corollaryCertificate()`. Reproduce: `swift test -c release -filter testTensorLUTFormalCorollaryCertificate`.

Theorem 10.2 (TensorLUT emitter / freeze corollary)**.** *Under Theorem 10.1 hypotheses plus emit-via-E and `mutatedPreserving`: if $\pi(w) = 0$ then $E(w)$ recovers the binary INIT; every freeze-preserving genotype remains a partial involution containing the frozen pairs.*

Still not melt completeness for arbitrary netlists. The named lemmas of Theorem 10.1 are `discretenessPenalty`, `cryptoFitnessMSE`, `combinedObjective`, `emitterThreshold`, `involutionSandwich`, `freezeMask`. Source path: `Sources/HELUTCore/TensorLUTFormal.swift`.

Honesty note 10.1. Theorem 1 does *not* prove recovery of arbitrary keys; a U-534 / P1030680 plaintext; or that melt is complete for all netlists. Shatter / hold grades remain empirical (**C8**, **C9**, **H6**).

Reproduced (C44). Theorem $1''$ melt–freeze–snap on a coordinate-separable Boolean interpolant: unique maximizer $w = t$, snap basin $|w_i - t_i| < 1/2$, wrong freeze blocks $F = 0$. `TensorLUTFormal.meltFreezeSnapCertificate()`. Reproduce: `swift test -c release -filter testTensorLUTMeltFreezeSnapCertificate`. Not multi-LUT topological melt.

Reproduced (C48). Two-LUT cascade $y = (a \wedge b) \oplus c$: melt from 0.5, snap, emit two LUT6 Verilog cells. Snapped INIT matches all 8 Boolean corners (a dual interpolant such as NAND+XNOR is allowed). `testTwoLUTCascadeMeltFreezeSnapEmit`. Not Yosys re-synth.

Theorem 10.3 (TensorLUT melt–freeze–snap, separable interpolant)**.** *Under a fully observed 1-LUT (each used INIT address an independent Boolean target t), $F(w) = -\|w - t\|_2^2 - \lambda\pi(w)$ has unique maximizer $w = t$; the emitter recovers t on the open cube $|w_i - t_i| < 1/2$; a freeze away from t makes $F = 0$ unreachable.*

10.4 Five-cell test for Theorem 1

Cell	Artifact
Proof	Theorem 10.1 + `TensorLUTFormalCertificate`
Table	Lemmas vs `holds` on the certificate; campaign grades in `BREAK_P1030680.md` §21
Metric	$F_{\text{crypto}} = 0$ on unmutated baseline; $\pi = 0$ on binary INIT
Examples	M4 baseline emit; freeze-core involution; blind 3-pair PASS
Application	Stecker search that cannot propose a non-reciprocal map

Exercise 10.1 (Read a certificate)**.** Run the reproduce filter (Appendix D) or, on the theory track, read `TensorLUTFormal.certificate()` and list the six `holds` bits. For each lemma, write one sentence a journalist may print and one they may not.

10.5 Shatter versus hold (empirical)

A full cold-start on a large sequential cipher discovers continuous shortcuts that *shatter* when λ rises: F_{crypto} looks perfect in float and dies on the Boolean cube. Targeted melting freezes known-good LUT blocks (`freezeMask`) and evolves only the unknown cone—clause 6 of Theorem 10.1 is why frozen coordinates do not pay π. Early resignation reboots doomed lineages rather than squeezing a reciprocal shortcut to the end.

Definition 10.1 (Grade)**.** Under a published λ schedule, a lineage **holds** if the emitted binary netlist preserves $F_{\text{crypto}} = 0$ (or a stated bar). It **shatters** if discreteness kills the continuous solution. Both grades are science. Shatter is not a failed demo; it is evidence about shortcuts. It is *not* a clause of Theorem 10.1.

Reproduced (C8, C9). TensorLUT M4 baseline $F_{\text{crypto}} = 0$ plus Verilog emit; Stecker involution sandwich with blind 3-pair PASS. Method claims, not a U-534 / P1030680 plaintext.

10.6 Involution sandwich

Stecker genotypes are partial involutions (disjoint pairs) by construction (Theorem 10.1, clause 5). The GA cannot propose a non-reciprocal map. The 16 cone LUTs around an identity plugboard cannot invent letter swaps by INIT melt; the sandwich injects $S(\text{CT})$ into a frozen core and scores soft PT against bits of $S(P)$. Reciprocity is structural.

Honesty note 10.2. Short cribs leave unused pairs unconstrained and can admit false $F = 0$ steckers. Grade active-map agreement on $\text{CT} \cup \text{PT}$, use parsimony, grow the pair budget on plateau, and allow soft freeze + thaw. TensorLUT grades are not a campaign decrypt (**N**, **H6**).

10.7 Adversarial synthesis as a compiler loop

Forward: silicon → uniform float tensors. Reverse: cooled tensors → INIT hex → Verilog LUT instantiations. The loop is a generative compiler pitting continuous optimization against discrete physical limits [8]. Theorem 10.1 is the invariant the loop is forced to obey. It is not a guarantee that the loop finds a key.

Exercise 10.2. Why must $\pi(w) = \sum w_i(1 - w_i)$ vanish on binary INITs, and why is a fitness that ignores π allowed to "solve" a cipher that no FPGA can implement? Give a two-LUT numerical cartoon. Which clause of Theorem 10.1 did you just unpack?

Exercise 10.3 (Seminar)**.** Theorem 10.1 is structural, not complete. Write the statement of a completeness theorem you wish were true (melt recovers X on netlist class Y under schedule Z), and the counterexample shape you would try first. Do not upgrade shatter/hold grades into that theorem.

Non-claim. Pillar II does not break P1030680, does not replace Welchman search, and does not make TensorLUT a PBS. Parallel research; campaign fitness stays cleartext. **C19** is a theorem about the objective and the involution contract, not about plaintext.

Part V

Pillar III — Adversarial Polymorphic Ciphers

Chapter 11

Ciphers that grow under fire

This section grows when. A polymorphic Red/Blue standard beyond the E256 SoftBus instance (roadmap: Schneier pillar), and IND-CPA / Red-pressure security proofs that go past the structural SoftBus contract. This edition teaches Theorem 2 (machine-checked reciprocity / fail-closed) plus the laboratory loop—not a finished industry standard.

11.1 Why a cryptanalysis lab must design ciphers

A course that only breaks historical machines trains curators. Pillar III asks Blue to sit in the same room as Red: mutate the non-linear feedback under adversarial pressure, and fail closed when the invariants break [2, 8].

The historical Enigma leaks this laboratory proved fatal—26-letter menus, self-stecker ban, thin plugboards, odometer stepping, paper day keys—are deleted on purpose in Enigma256 (E256). The reciprocal rotor *contract* is kept: encrypt ≡ decrypt under the same machine state.

11.2 The E256 contract, for lecture

- Alphabet is a byte, not 26 letters.
- Plugboard is a 256-entry involution, 128 pairs, no fixed points.
- Four active rotors drawn from a 16-rotor day pool; un-reflector allows fixed points.
- After each byte, NLFF step enables advance offsets; a Galois LFSR clocks. Not an odometer.
- Control-plane crypto (KEM, HKDF, AEAD, signatures) never enters datapath BRAMs.
- The live field is Apple Silicon SoftBus, not an FPGA board on the critical path.

Reproduced (C10). Enigma256 reciprocity, fail-closed behavior, and bijection tests. See `Enigma256.md` and fixtures.

Reproduced (C24). Enigma256 SoftBus Theorem 2: five machine-checked lemmas (bijection, reciprocity, stream round-trip, day-key involutions, `coupledCubic6` reject). Statement: `directives/enigma256-theorem.md`.

Reproduced (C39). One frozen 8-bit E256 scramble round (algebraic sboxes + UKW, offsets 0) under GGSW Metal SING at N=1024. Not live BRAM tables, not NLFF, not a full SoftBus burst.

11.3 Theorem 2 (structural SoftBus)

Theorem 11.1 (Enigma256 SoftBus reciprocity / fail-closed)**.** *Under the hypotheses of* `Enigma256Formal.certifica` *(reciprocal scramble path; day-key involution builders; NLFF retaps do not rewrite frozen scramble): frozen scramble is a permutation and an involution; stream encrypt-then-decrypt recovers plaintext under identical keys; derived plugboard is fixed-point-free involution and un-reflector is an involution;* `hardenedCubic()` *rejects* `coupledCubic6`*.*

Machine-checked: `Enigma256Formal.certificate()` / `testEnigma256FormalCertificate`. Reproduce: see Lab appendix / `REPRODUCE.md`.

Honesty note 11.1. Theorem 2 is a *structural SoftBus contract*—not IND-CPA, not a claim that Red TensorLUT/KPA/`ent` cannot force Blue generation rolls, and not a finished polymorphic-cipher standard.

11.4 Red pressure

Red is not a separate team with a PDF of attacks. Red is TensorLUT cones, SoftBus known-plaintext attacks, and `ent` on the keystream—on the same Mac that rolls Blue generations.

A generation that cannot hold under that pressure does not ship. Fail-closed means: if reciprocity, bijection, or a stated invariant breaks, the roll aborts rather than emitting a "mostly fine" Verilog. Theorem 2 covers the structural abort path for coupled NLFF harden; empirical Red batteries remain separate grades.

11.5 What students design

Exercise 11.1 (Blue studio)**.** Propose one additional invariant E256 should fail-closed on (example: NLFF period lower bound; stecker-free involution; BRAM burst width). Write the test that would refuse a generation. Do not implement a new cipher in this edition unless the instructor opens a project slot.

Exercise 11.2 (Historical leaks as a checklist)**.** Map each deleted Enigma leak to the E256 construct that replaces it. One table, six rows. Then add a seventh row: a leak this design may still have, labeled as a question, not a claim.

11.6 Relation to Pillars I and II

E256 is not evaluated as torus FHE in this edition's lecture arc. It is a SoftBus datapath with a Red TensorLUT arm. Putting E256 on the encrypted HELUT clock is a possible future lab, and a possible **N** if someone calls that lab "production messaging."

Non-claim. E256 is not a NIST candidate in this book, not a replacement for TLS, and not an implication that polymorphic rotors beat lattices. It is a pedagogical and laboratory standard for fail-closed mutation.

Part VI

Laboratory, Applications, and the Frontier

Chapter 12

Application gallery

Applications exist to keep pillars honest. A compiler with no netlist is a speech; a melt with no cipher is a float toy; a polymorphic cipher with no Red is a blog post.

Canonical nine-slot card: `directives/application-gallery.md`.

12.1 Pillar I (encrypted)

1. **Encrypted `full_adder` SING.** Multi-LUT correctness at production-shaped N (**C6**, **C20**, **C21**).
2. **Encrypted tree / regex SING.** Compiler is not adder-specialized (**C6**, demo N).
3. **Hardness + noisy-BK certificates.** Estimator fill-in and covering-gadget residuals (**C5**, **C22**, **C23**). Product-shaped N=1024 inject is a graded failure (**C26**); Metal SING still noiseless BK (**H4**).
4. **Encrypted sequential.** Counter (**C38**), toy `NOP`/`ADD` (**C40**), PicoRV32 `addi`+`sw`+`lw` at demo N=8 (**C50**); Metal NOP-fetch (**C51**).
5. **4-bit CSA vs ripple LUT cut** (**C42**). Architecture, not TensorLUT melt.

12.2 Pillar I shape laboratory (oracle — not FHE)

1. **Decision tree.** A 4-bit non-linear boundary, 7 LUTs. Exact threshold logic on a batch of records. Teaches: non-linear Boolean $\neq$ neural net, and B is a capacity knob.
2. **Regex / DFA search.** 3-character matcher, 23 LUTs. Large B turns unified memory into the story.
3. **PicoRV32.** $\sim$4.8k LUTs, $\sim$1.5k DFFs. Scripted `resetn` boot. Oracle path: **C1**. Encrypted 1-tick at demo N=8: **C50** `lw` sees 1; Metal NOP-fetch **C51**.

12.3 Pillar II

1. **M4 TensorLUT baseline emit** (**C8**).
2. **Stecker involution sandwich / formal** (**C9**, **C19**, **C25**, **C44**, **C48**).

3. **Shatter vs hold under λ** — seminar empirics; not a decrypt (**H6**).

12.4 Pillar III

1. **SoftBus reciprocity / bijection** (**C10**, **C24**).
2. **Encrypted 1-round frozen scramble** (**C39**) — algebraic slice under torus FHE, not live BRAM.
3. **Red battery** (TensorLUT / KPA / `ent`) — empirical grades in `logs/enigma256-*`.
4. **Fail-closed NLFF harden** (**C24** clause 5).

12.5 Campaign as a case study (not a decrypt)

Welchman-path break of known P1030684 (**C2**), $\leq$ 10-plug SAT kill chain (**C3**), Potsdam/Plaice $\neq$ P1030680 (**C11**), windowed discriminator versus whole-message turnover (**C12**): these belong in a cryptanalysis seminar attached to Pillar II methods.

Non-claim. This book does not claim a decrypt of P1030680. Campaign fitness is cleartext Metal batch, not HELUT encrypted tick rate.

Exercise 12.1. Pick one application tier and write a five-cell card for it. If you cannot name two runnable instances, it is not ready for the gallery lecture.

This section grows when. Figures for the site gallery; an encrypted tree/regex Metal SING table at production N; any new application that earns a **C** row.

Chapter 13

Laboratories

Systems-track labs assume macOS 14+, Apple Silicon, Swift 6.3, and a release `helut` at repo root: `swift build -c release -product helut`. Theory-track substitutes are stated per lab.

13.1 Lab 0 — Claim literacy (everyone)

Read `directives/claim-sheet.md` at the printed epoch. Classify five public sentences (instructor-provided) as **C**, **H**, **N**, or frontier. No machine required.

13.2 Lab 1 — Negacyclic oracle

Implement $N = 8$ schoolbook mul in $\mathbb{Z}/2^{32}\mathbb{Z}[X]/(X^N + 1)$ and check a dense Toeplitz expansion against it on torus corners. Theory track: $N = 4$ by hand (Chapter 6 exercises).

13.3 Lab 2 — Netlist clock

Compile `counter` or `full_adder` Yosys JSON with `-compile-only`. Tick with `-bench`. Identify B, N, LUT count, DFF count. Explain in writing why N is not the circuit size.

13.4 Lab 3 — Boolean oracle versus FHE path

```
.build/release/helut --bench netlist.json --degree 64 \
  --lut-backend pbs --encoding phase --bench-equiv
.build/release/helut --bench netlist.json --degree 8 \
  --bench-encrypted --cpu-only --sing --vectors 8
.build/release/helut --bench counter_netlist.json --degree 8 \
  --bench-encrypted --cpu-only --sing --vectors 8
.build/release/helut --bench toy_isa_netlist.json --degree 8 \
  --bench-encrypted --cpu-only --sing --vectors 32
.build/release/helut --bench picorv32_netlist.json --degree 8 \
  --bench-encrypted --cpu-only --sing --ticks 48 --reset-hold 3 \
  --encrypted-mem prog --paths 'blind-rotate public-ms boolean'
```

Deliverable: a table with columns {backend, N, SING, certificates printed, may we say "encrypted" on a slide?}.

13.5 Lab 4 — Production-shaped SING

Reproduced (C6, H2 closed). full_adder encrypted SING through $N = 1024$.

Follow Appendix D. Record ms/row at $N = 8$ and $N = 1024$. Write the **H1** asterisk next to hardness. Write the **H4** asterisk next to depth: **C22** measured covering gadget; $N = 1024$ SING is still $e = 0$ BK.

Theory track: parse a published SING log and reconstruct the table without running Metal.

13.6 Lab 4b — Measured noisy BK (C22)

```
.build/release/helut --measure-bk-noise --degree 8 \
  --trials 8 --bk-noise 64
```

Deliverable: B_{bk}, $\hat{\sigma}$, $\delta/2$, and one sentence on why $\ell = 1$ `booleanPublicMS` is omitted. Theory track: read `logs/helut-noisy-bk-measure.log`.

13.7 Lab 5 — Metal compiler post-mortem

Compare fused-megagraph DNF, **C16**–**C21** using Chapter 8's table. Optional systems: `make test-metal-p1` and one `-bench-encrypted-micro -degree 64` run. Deliverable: the **H3** paragraph in your own words (scheduler vs NTT).

13.8 Lab 6 — TensorLUT Theorem 1 (C19)

```
swift test -c release --filter testTensorLUTFormalCertificate
```

Deliverable: name the six lemmas, the certificate hypotheses, and one sentence that Theorem 10.1 does *not* prove. Optional: `testTensorLUTMeltFreezeSnapCertificate` (**C44**) — what class of INIT genomes the unique-maximizer lemma covers. Theory track: read `Sources/HELUTCore/TensorLUTFormal.swift` and `directives/tensorlut-theorem.md`; reconstruct the five-cell card without running Swift.

13.9 Lab 7 — Five-cell audit (everyone)

The signature assignment (Chapter 2). One abstract sentence from `paper/helut.tex`; five cells; a pass/fail on whether the sentence is lecture-safe at this epoch.

13.10 Lab 8 — Frontier proposal (everyone)

Chapter 15: pick one avenue. Propose the first experiment that would mint a **C** row. You may not claim the avenue works.

13.11 Optional project slots

TensorLUT involution sandwich replay (**C9** logs), E256 invariant studio, or a new Yosys netlist (student RTL → SING). Instructor approval required; campaign decrypt projects are out of scope.

This section grows when. Autograded SING harnesses for course VMs; a Dockerized theory track; non-Apple ring-oracle labs.

Chapter 14

Open problems (trajectory)

This chapter is the discovery path after disclosure [9]. None of these items are lecture facts until they have receipts.

14.1 Near term

Track	Status at epoch 2026-08-13 / C51	Next experiment
H2 full_adder $N \geq 256$	**Closed** (Z_{2N} pack / `rotationPower`)	Keep as a worked bug in Chapter 6.
H1 Sage lattice-estimator	**C23** filled; production $\lvert\Delta\rvert = 4.5$; core-SVP vs Cost `rop` divergences	Optional retune / quote estimator-only on $\Delta > 16$ rows.
H3 Metal BR at large N	**C20** boolean 10.6 s; **C21** crypto $\ell = 2$ 11.38 s	NTT inside crypto $\ell = 2$ at N=1024 (incomplete public-MS gadget).
H4 Noisy BK	**C43** k=4 SING @ N=1024 σ=128; ε 4-trial -43	k in $(4, 8)$ or public-ms at k=8; native k=1 still open.
Encrypted sequential	**C51** Metal PicoRV32 NOP-fetch @ demo N=8 (7.94 s/row)	Metal N=1024; fused metal-netlist.
Campaign catalog	Middle ring $\neq A$ untested; catalog parked @417	Resume `-bombe-from 418`.
Garble / quarantine	Soft-band grades	Sister-message lessons; not a decrypt claim.

14.2 Mid-term pillars

1. **Pillar I science:** shallower nets, NTT/persist graphs, estimator-backed params (this book's Parts III).
2. **Pillar II science:** Theorem 1 landed (**C19**); corollary emitter / freeze (**C25**); melt–freeze–snap on a separable interpolant (**C44**); 2-LUT cascade emit (**C48**); remaining: multi-LUT topological melt, stream-cipher melts.
3. **Pillar III science:** SoftBus Theorem 2 landed (**C24**, Chapter 11); remaining: polymorphic Red/Blue standard beyond E256 SoftBus.
4. **FHE gate / ZK depth:** TensorLUT aimed at multiplicative depth (queued; not a part of this edition).

5. **Side-channel (parked):** bgpucap-style power on live Metal graphs only with controlled fixtures — Chapter 15, avenue 5.

Exercise 14.1 (Pick a next experiment)**.** Choose one open hedge. Write a one-page lab proposal: command line, pass bar, which box in this book would change, and which **N** you refuse to imply if the experiment fails.

This section grows when. Each closed row in this table deletes a hedge box elsewhere and adds a reproduced box. That is the only way this chapter shrinks.

Chapter 15

Frontier (not claims)

The text in this chapter is adapted from the project's speculative avenues [7]. It is included so the public story stays honest: **reproducible core now; weird frontier labeled as frontier.**

Instructors: last two seminars, or a 10% essay. Never a true/false bank.

15.1 Encrypted LLM guardrails

Frontier — not a claim. Evolve a tiny neural or logic-gate firewall in the encrypted domain that sits between an untrusted prompt and a local model. The firewall circuit would evaluate in the dark on token tensors and mutate its Boolean topology from adversarial inputs it blocked—without decrypting prompt, weights, or mutation rules in memory.

First graduating experiment (suggested): a 4-LUT encrypted classifier SING on synthetic token bits, with a certificate, *before* any talk of genetic mutation or Ollama.

15.2 Self-modifying encrypted binaries

Frontier — not a claim. A PicoRV32 that already ticks on an `MPSGraph` under oracle encodings is not a dark VM. The avenue is a binary that rewrites its own instructions while remaining FHE-encrypted, so the host sees tensor multiplies and cannot inspect the guest.

First graduating experiment: encrypted instruction-fetch SING on a toy ISA smaller than PicoRV32, with a written ethics note. This is a privacy architecture problem, not a malware homework.

15.3 Honey-token ledgers

Frontier — not a claim. Batched encrypted pattern search composed with a zero-knowledge ledger, so a sync evaluates trapdoor circuits without the storage node learning which file or which trigger fired.

First graduating experiment: regex netlist SING at modest B with an explicit non-claim that the node learns nothing—then measure what the *host process* still sees (lengths, timings). Honesty before mesh networks.

15.4 Genetic hardware synthesis

Frontier — not a claim. `fhe-evolve` mutates truth tables and connectivity; Yosys recompiles; `MPSGraph` re-lowers; the running system processes a stream through hardware it just invented.

First graduating experiment: one offline generation of a 2-LUT netlist that SING-matches a fixed spec, wall-clock of synth+lower, no on-the-fly claims.

15.5 Metal-tick power analysis (parked)

Frontier — not a claim. Treat the live Metal FHE pipeline as the device under test. Capture GPU-side power only while a tick-synchronized graph runs *controlled fixtures*. Classical DPA/CPA against aligned traces. A distinguish or a clean negative are both science. Opportunistic capture of arbitrary workloads is out of scope.

First graduating experiment: fixture harness that emits tick markers and a published "no analysis yet" log. Do not assign attacks against third-party graphs.

15.6 Rules for frontier essays

1. Label every paragraph frontier or first-experiment.
2. Name the **C** row it would become, and the **N** it must not imply on the way.
3. Cite Chapter 3. If the five cells cannot even be *imagined*, the essay is not ready.

Part VII

Appendices

Appendix A

Notation

Symbol	Meaning
q	Torus modulus; HELUT uses $q = 2^{32}$.
N	Polynomial degree / ciphertext shape. Not LUT count.
n	LWE dimension (often $n = N$ in HELUT boolean production).
B	Batch axis: independent copies of one netlist.
δ	$q/(2N)$, rotation / message spacing.
σ	LWE Gaussian width.
ε	Ingest failure probability (Gaussian certificate).
B_{bk}	BK noise bound after BR; noiseless (0) on default Track A throughput SING; measured under covering gadgets (**H4**, **C22**/**C36**).
ℓ	GGSW gadget levels.
W	CMUX tile width in the Metal compiler.
R_q	$\mathbb{Z}_q[X]/(X^N + 1)$.
M_A	Negacyclic Toeplitz matrix of polynomial A.
w	TensorLUT INIT genome in $[0, 1]^{64L}$.
L	Number of LUT6 cells in a TensorLUT netlist.
$y(w)$	Multilinear extension of INITs (exact on $\{0, 1\}$).
t	Target bit vector for F_{crypto}.
$\pi(w)$	Discreteness penalty $\sum_i w_i(1 - w_i)$.
$E(w)$	Emitter $\mathbf{1}[w \geq \frac{1}{2}]$.
M	Freeze mask: coordinates excluded from π.
λ	Discreteness squeeze on $\pi(w)$; Theorem 1 needs $\lambda \geq 0$.
`$lut`	Yosys look-up cell; one programmable gate.
SING	**S**timuli **In**, **G**olden out: encrypted evaluation ≡ cleartext netlist on the same generic input vectors (`-sing`); certificate lines ride with the tick.
Cn, **H**n, **N**n	Claim, hedge, non-implication IDs.

Appendix B

Claim index (snapshot)

Canonical living inventory: `directives/claim-sheet.md`. If this appendix disagrees with the sheet, the sheet wins. Snapshot epoch: 2026-08-13 / C51.

B.1 Reproducible results

ID	Result
C1	Yosys `$lut`/DFF → Metal/CPU tensor eval (oracle).
C2	Welchman path breaks known P1030684 end-to-end.
C3	≤ 10-plug / SAT kill chain removes ghosts.
C4	Encrypted LWE/GLWE + GGSW BR per `$lut`.
C5	Certificates on encrypted ticks.
C6	Encrypted ≡ clear multi-netlist SING; adder $N \leq 1024$.
C7	Metal 1-LUT BR microbench (fused ancestor numbers).
C8	TensorLUT M4 baseline $F_{\text{crypto}} = 0$ + Verilog emit.
C9	Stecker involution sandwich; blind 3-pair PASS.
C10	Enigma256 reciprocity; fail-closed / bijection.
C11	Potsdam/Plaice keys ≠ P1030680.
C12	Windowed discriminator vs whole-message turnover flaw.
C13	Metal tiled-kernel BR at $N = 1024$ (pre-fusion lineage).
C14	Metal full_adder SING at $N = 1024$ (pre-NTT lineage).
C15	Metal netlist-scheduled SING at $N = 1024$.
C16	Metal fused EP kernel; 1.043 s/BR at $N = 1024$.
C17	GPU-resident BR tile; 0.519 s/BR; SING 12.2 s/8.
C18	3-prime NTT persist BR; 0.433 s/BR; SING 15.6 s/8.
C19	TensorLUT continuous→discrete Theorem 1 (six machine-checked lemmas).
C20	Wavefront-parallel independent `$lut` BRs; boolean SING 10.6 s/8.
C21	Metal cryptoPublicMS $\ell = 2$ SING at $N = 1024$; 11.38 s/8.
C22	Measured noisy-BK residual → $B_{\text{bk}}/\hat{\sigma}$ (covering gadget).
C23	Native Sage lattice-estimator fill-in; production $\lvert\Delta\rvert = 4.5$.
C24	Enigma256 SoftBus Theorem 2 (reciprocity / fail-closed; five lemmas).
C25	TensorLUT Theorem 1 corollary (emitter–discrete + involution under freeze).
C26	Noisy-BK identity residual at N=1024: inject B=64 undecodable; B=4 $\varepsilon \log_2 \approx -1$.
C27	Exact public-MS covering under q=2^{32} only at $N \in \{8, 128\}$ (Theorem 3).
C28	Track B Metal cryptoPublicMS SING at N=128 with noisy BK (B=64) PASS.

Continued on next page

ID	Result *(continued)*
C29	Exact public-MS covering stays $\{8, 128\}$ for any power-of-two $q{=}2^w$ (Theorem 3′).
C30	Track B ε vs inject B at $N{=}128$: printed ε OK at small B; $\hat{\sigma}$ still $\sim 10^4$–10^5.
C31	Incomplete public-MS gap at $N{=}1024$: uncoveredBits= 10; approx candidate =`.crypto`.
C32	Track A approx: `.crypto` inject $B{=}1$ at $N{=}1024$ decodable; $\varepsilon \log_2 \approx -8.4$ at 8 trials (not -64).
C33	Track A Metal SING covering `.crypto` at $N{=}1024$ with $B{=}1$ PASS (secret + public-ms).
C34	Track A covering baseLog= 4: $\varepsilon \log_2 \approx -913$ ($\leq 2^{-64}$) + Metal SING PASS.
C35	Track A covering baseLog= 2: $\varepsilon \leq 2^{-64}$ through inject $B{=}16$ ($\varepsilon \log_2 \approx -65.4$) + Metal SING PASS.
C36	Track A covering baseLog= 1: $\varepsilon \leq 2^{-64}$ through inject $B{=}32$ ($\varepsilon \log_2 \approx -139$) + Metal SING PASS; $B{=}128$ still fails.
C37	Track A Gaussian BK inject on covering-b1: $\sigma{=}24$ meets $\varepsilon \leq 2^{-64}$ + Metal SING PASS; torus $\sigma{\approx}128$ still undecodable.
C38	Encrypted sequential DFF host-clock: 4-bit counter Metal $N{=}1024$ SING PASS (15.88 s/4). Not PicoRV32.
C39	Encrypted 1-byte/1-round E256 frozen scramble Metal $N{=}1024$ SING PASS (22.77 s/2). Not live BRAM.
C40	Encrypted toy NOP/ADD ISA Metal $N{=}1024$ SING PASS (20.31 s/4). Not PicoRV32.
C41	Covering-b1 Gaussian $\sigma{=}128$ meets $\varepsilon \leq 2^{-64}$ at $N \leq 512$ + Metal SING; $N{=}1024$ still fail.
C42	4-bit CSA vs ripple: LUT2 11 $\rightarrow$ 8 (-27%) + encrypted SING PASS. Not TensorLUT melt.
C43	$k\delta$ encoding at $N{=}1024$: $k{=}4$ Metal SING PASS on covering-b1 $\sigma{=}128$; ε bar not stable at 4 trials.
C44	TensorLUT melt–freeze–snap on a separable Boolean interpolant (three lemmas). Not arbitrary-netlist melt.
C45	Encrypted PicoRV32 1-tick SING at demo $N{=}8$: 76.12 ms/row; 4-bit hardness. Not production N.
C46	Encrypted PicoRV32 10-tick `resetn` boot SING at $N{=}8$: 0.972 s/10; $Q \equiv$ clear. Plain `$_DFF_P_` native-D fix.
C47	Encrypted PicoRV32 NOP-fetch: 10 instruction fetches, PC $0{+}4k$ through $0x24$; 3.15 s/32.
C48	2-LUT cascade melt–snap–emit $(a \wedge b) \oplus c$; 8-corner SING of snapped INIT (dual interpolant OK).
C49	Encrypted PicoRV32 `addi`+`sw`: store `wdata`=1, RAM0=1; 3.79 s/48 @ demo $N{=}8$. Load-back is **C50**.
C50	Encrypted PicoRV32 `lw` sees stored 1 (NBA xfer); 3.77 s/48 @ demo $N{=}8$. Not Metal PicoRV.
C51	Metal PicoRV32 NOP-fetch @ demo $N{=}8$: tiled BR, 2 fetches, 63.55 s/8. Not fused netlist / not $N{=}1024$.

B.2 Open hedges

ID	Asterisk
H1	**C23** filled JSON. Production $\|\Delta\| = 4.5$; divergences are core-SVP vs Cost `rop`.
H2	**Closed** 2026-08-12 (Z_{2N} pack). Kept as history.
H3	**C20**/**C21** SING bars met; serial NTT SING still loses vs persist.
H4	**C43** k=4 SING at N=1024 σ=128; $\varepsilon \leq 2^{-64}$ not stable. Native k=1 still **C37**.
H5	`*PublicMS` gadgets ($g_0 = \delta$): on-lattice intent, not a closer of old **H2**.
H6	TensorLUT / quarantine vs campaign: parallel research, not P1030680 PT.
H7	Catalog / Regenbogen / UEBUNG: negatives graded; middle ring $\neq A$ untested; catalog parked @417, resume `-bombe-from 418`.

B.3 Standing non-implications

Do not let prose imply: new lattice assumption; "we invented TFHE"; production keys without estimator; mock-torus = FHE; P1030680 decrypted; campaign fitness = encrypted tick rate; TensorLUT = U-534 break; side-channels measured; quantum attacks analyzed. Wild ideas in Chapter 15 are trajectory, not results.

Appendix C

Fifteen-week syllabus

Suggested title: *Reconfigurable Homomorphic Computing.* 3 credits. Mix of lecture, seminar, and lab (Chapter 2).

Wk	Part	Classroom
1	I	Subject definition; living boxes; claim audit assigned.
2	II	Netlists, Yosys `$lut`, B vs N; Lab 1.
3	II	Torus, LWE, PBS sketch; $N = 4$ Toeplitz.
4	III	Pillar I pipeline; oracle vs encrypted; Lab 2.
5	III	SING; Z_{2N} pack as a worked bug; Lab 3.
6	III	Metal compiler Phase 1/2; fused DNF as a case.
7	III	Certificates; hardness table; Lab 4.
8	—	Midterm (foundations + five-cell); Lab 5 optional.
9	IV	TensorLUT setup; Theorem 1 (**C19**); Lab 6.
10	IV	Shatter vs hold; involution sandwich; campaign as case study (no decrypt).
11	V	E256 contract; Red/Blue; fail-closed.
12	VI	Application gallery; student five-cell cards.
13	VI	Labs catch-up; claim-audit drafts due.
14	VI	Open hedges; pick-a-next-experiment.
15	VI	Frontier seminars; essays due; living-epoch recap.

Final: take-home claim audit plus a short oral defense of one reproduced box and one hedge box. If the student upgrades the hedge, they have not passed the course's signature skill.

Appendix D

Reproduce commands (course subset)

Canonical file: `REPRODUCE.md`. Cookbook: `directives/parameter-cookbook.md`. Build:

```
swift build -c release --product helut
```

D.1 Documents

```
make textbook # this book
make paper # three-pillar report
make writeup # P1030680 campaign report
```

D.2 Hardness (C23, H1)

```
.build/release/helut --hardness-table
./Scripts/helut_sage_estimate.sh
```

Native SageMath 10.9 at `~/Applications/SageMath-10-9.app`. Production row: HELUT 175.7 vs estimator 180.2. Four of eight anchors exceed the 16-bit merge tolerance — do not quote 176 as estimator cost on every row.

D.3 Encrypted SING (C4–C6, C38–C40)

```
.build/release/helut --bench netlist.json --degree 8 \
  --bench-encrypted --cpu-only --sing --vectors 8
./Scripts/helut_encrypted_sing.sh
.build/release/helut --bench counter_netlist.json --degree 8 \
  --bench-encrypted --cpu-only --sing --vectors 8
.build/release/helut --bench e256_round1_netlist.json --degree 16 \
  --bench-encrypted --cpu-only --sing --vectors 32
.build/release/helut --bench toy_isa_netlist.json --degree 8 \
  --bench-encrypted --cpu-only --sing --vectors 32
```

C38 counter, **C39** frozen E256 round (LUT4, $N \geq 16$), **C40** toy ISA. None of these is PicoRV32 or live BRAM SoftBus.

D.4 Metal microbench and NTT (C18)

```
make test-metal-p1
.build/release/helut --bench-encrypted-micro --degree 1024 \
  --trials 2 --warmup 1
```

Default at $N > 64$ is tiled-kernel with inlined NTT. Do not assign `-metal-br-fused` at $N = 1024$ as a timed lab.

D.5 Wavefront SING (C20, C21)

Boolean 10.6 s/8; crypto $\ell = 2$ 11.38 s/8 at $N = 1024$. Logs already in `logs/` — students need not re-time Metal unless the instructor assigns a machine-local check.

```
.build/release/helut --bench netlist.json --degree 1024 \
  --bench-encrypted --sing --vectors 8 \
  --paths 'blind-rotate-metal public-ms boolean'
.build/release/helut --bench netlist.json --degree 1024 \
  --bench-encrypted --sing --vectors 8 \
  --paths 'blind-rotate-metal public-ms crypto'
```

D.6 Measured noisy BK (C22)

Covering gadget only. $\ell = 1$ `booleanPublicMS` cannot carry BK noise.

```
.build/release/helut --measure-bk-noise --degree 8 \
  --trials 8 --bk-noise 64
.build/release/helut --measure-bk-noise --degree 128 \
  --trials 4 --bk-noise 64
```

D.7 Product-shaped N=1024 residual (C26)

Graded negative for production noisy BK (H4). Expect undecodable inject B=64; B=4 on `.crypto` may pass ∞-norm with $\varepsilon \log_2 \approx -1$ (not -64).

```
.build/release/helut --measure-bk-noise --degree 1024 \
  --trials 2 --bk-noise 64
.build/release/helut --measure-bk-noise --degree 1024 \
  --trials 2 --bk-noise 4
```

D.8 Exact public-MS covering (C27)

```
swift test -c release --filter testGGSWPublicMSCoveringCertificate
```

Exact degrees among $\{8, \ldots, 2048\}$: $\{8, 128\}$ only — for any power-of-two word w (**C29**), not just q=2^{32}. Statement: `directives/ggsw-public-ms-covering.md`.

D.9 Track B ε vs inject B (C30)

```
for B in 1 2 4 8 16 32 64; do
  .build/release/helut --measure-bk-noise --degree 128 \
    --trials 8 --bk-noise $B
done
```

Residual amplification stays large; printed $\varepsilon \log_2 \approx -24$ only at B=64.

D.10 Incomplete public-MS gap / Track A approx (C31/C32)

```
swift test -c release --filter testGGSWIncompleteCoveringCertificate
.build/release/helut --measure-bk-noise --degree 1024 \
  --trials 4 --bk-noise 1
```

uncoveredBits(1024) = 10. `.crypto` B=1 $\Rightarrow \varepsilon \log_2 \approx -8.4$ (8 trials). Statement: `directives/ggsw-incomplete-`

D.11 Track A Metal covering-crypto SING (C33)

```
.build/release/helut --bench netlist.json --degree 1024 \
  --bench-encrypted --sing --vectors 2 --bk-noise 1 \
  --paths 'blind-rotate-metal secret crypto'
.build/release/helut --bench netlist.json --degree 1024 \
  --bench-encrypted --sing --vectors 2 --bk-noise 1 \
  --paths covering-crypto
```

Expect PASS; covering `.crypto`, not `cryptoPublicMS`.

D.12 Track A covering-b4 $\varepsilon \leq 2^{-64}$ (C34)

```
.build/release/helut --measure-bk-noise --degree 1024 \
  --trials 8 --bk-noise 1 --covering-base-log 4
.build/release/helut --bench netlist.json --degree 1024 \
  --bench-encrypted --sing --vectors 2 --bk-noise 1 \
  --paths covering-b4
```

Expect $\varepsilon \log_2 \approx -913$; Metal PASS. Torus-scale B=128 still fails.

D.13 Track A covering-b2 $\varepsilon \leq 2^{-64}$ through B=16 (C35)

```
.build/release/helut-bench --measure-bk-noise --degree 1024 \
  --trials 4 --bk-noise 16 --covering-base-log 2
.build/release/helut-bench --bench netlist.json --degree 1024 \
  --bench-encrypted --sing --vectors 2 --bk-noise 16 \
  --paths covering-b2
```

Expect $\varepsilon \log_2 \approx -65.4$ at B=16; Metal PASS. B=32 fails the bar.

D.14 Track A covering-b1 $\varepsilon \leq 2^{-64}$ through B=32 (C36)

```
.build/release/helut-bench --measure-bk-noise --degree 1024 \
  --trials 4 --bk-noise 32 --covering-base-log 1
.build/release/helut-bench --bench netlist.json --degree 1024 \
  --bench-encrypted --sing --vectors 2 --bk-noise 32 \
  --paths covering-b1
```

Expect $\varepsilon \log_2 \approx -139$ at B=32; Metal PASS. B=128 still fails.

D.15 Track A σ=128 covering-b1 at $N \leq 512$ (C41)

```
.build/release/helut --measure-bk-noise --degree 512 --trials 4 \
  --bk-noise-sigma 128 --covering-base-log 1
.build/release/helut --bench netlist.json --degree 512 \
  --bench-encrypted --sing --vectors 2 --bk-noise-sigma 128 \
  --paths covering-b1
```

Does not close N=1024 at native k=1. Headroom is $\delta = q/(2N)$.

D.16 $k\delta$ encoding at N=1024 (C43)

```
.build/release/helut --measure-bk-noise --degree 1024 --trials 4 \
  --bk-noise-sigma 128 --covering-base-log 1 --boolean-scale-mul 4
.build/release/helut --bench netlist.json --degree 1024 \
  --bench-encrypted --sing --vectors 2 --bk-noise-sigma 128 \
  --paths covering-b1 --boolean-scale-mul 4
```

k=4 Metal SING PASS; 4-trial $\varepsilon \log_2 \approx -43$ (not -64). k=8 meets ε; public-ms SING fails.

D.17 Track B Metal SING with noisy BK (C28)

```
.build/release/helut --bench netlist.json --degree 128 \
  --bench-encrypted --sing --vectors 4 --bk-noise 64 \
  --paths 'blind-rotate-metal public-ms crypto'
```

Covering degree. Expect PASS and non-zero decodable B_{bk}.

D.18 TensorLUT Theorem 1 (C19)

```
swift test -c release --filter testTensorLUTFormalCertificate
```

Six lemmas must `hold`. Statement: `directives/tensorlut-theorem.md`. Not a U-534 / P1030680 decrypt.

D.19 Enigma256 SoftBus Theorem 2 (C24)

```
swift test -c release --filter testEnigma256FormalCertificate
```

Five lemmas must `hold`. Statement: `directives/enigma256-theorem.md`. Structural SoftBus contract—not IND-CPA.

D.20 TensorLUT Theorem 1 corollary (C25)

```
swift test -c release --filter testTensorLUTFormalCorollaryCertificate
```

Emitter–discrete agreement and involution-under-freeze must `hold`. Statement: `directives/tensorlut-theorem` (corollary).

D.21 TensorLUT melt–freeze–snap (C44)

```
swift test -c release --filter testTensorLUTMeltFreezeSnapCertificate
```

Three lemmas must `hold` (unique maximizer, snap basin, freeze). Statement: `directives/tensorlut-theorem.m` (Theorem 1″). Not multi-LUT topological melt.

D.22 Encrypted PicoRV32 1-tick (C45)

```
.build/release/helut --bench picorv32_netlist.json --degree 8 \
  --bench-encrypted --cpu-only --sing --vectors 1 \
  --paths 'blind-rotate public-ms boolean' \
  | tee logs/helut-encrypted-n8-cpu-sing-picorv32.log
```

Expect PASS, $\sim$76 ms/row, hardness 4.0 bits (demo N). Output SING; Q SING is **C46**.

D.23 Encrypted PicoRV32 10-tick boot (C46)

```
.build/release/helut --bench picorv32_netlist.json --degree 8 \
  --bench-encrypted --cpu-only --sing --ticks 10 --reset-hold 3 \
  --paths 'blind-rotate public-ms boolean' \
  | tee logs/helut-encrypted-n8-cpu-sing-picorv32-boot10.log
```

Expect PASS 0.972 s/10, $Q \equiv$ clear. Idle memory. LUT-tax: N=32 $\sim$819 ms; N=64 $\sim$4.35 s.

D.24 Encrypted PicoRV32 NOP-fetch (C47)

```
.build/release/helut --bench picorv32_netlist.json --degree 8 \
  --bench-encrypted --cpu-only --sing --ticks 32 --reset-hold 3 \
  --encrypted-mem nop --paths 'blind-rotate public-ms boolean' \
  | tee logs/helut-encrypted-n8-cpu-sing-picorv32-nop-fetch.log
```

Expect 10 fetches, addresses $0, 4, \ldots,$ `0x24`, $Q \equiv$ clear.

D.25 Encrypted PicoRV32 addi+sw (C49)

```
.build/release/helut --bench picorv32_netlist.json --degree 8 \
  --bench-encrypted --cpu-only --sing --ticks 48 --reset-hold 3 \
  --encrypted-mem prog --paths 'blind-rotate public-ms boolean' \
  | tee logs/helut-encrypted-n8-cpu-sing-picorv32-prog-store.log
```

Expect STORE `wdata`=1, RAM0=1, 14 fetches, $\sim$79 ms/row.

D.26 Encrypted PicoRV32 lw sees 1 (C50)

```
.build/release/helut --bench picorv32_netlist.json --degree 8 \
  --bench-encrypted --cpu-only --sing --ticks 48 --reset-hold 3 \
  --encrypted-mem prog --paths 'blind-rotate public-ms boolean' \
  | tee logs/helut-encrypted-n8-cpu-sing-picorv32-prog-lw.log
```

Expect LOAD xfer `rdata`=1, STORE `wdata`=1, 14 fetches, $\sim$79 ms/row, $Q \equiv$ clear. Demo N=8.

D.27 Metal PicoRV32 NOP-fetch (C51)

```
.build/release/helut --bench picorv32_netlist.json --degree 8 \
  --bench-encrypted --sing --ticks 8 --reset-hold 3 \
  --encrypted-mem nop --paths 'blind-rotate-metal public-ms boolean' \
  --metal-br-tile 8 \
  | tee logs/helut-encrypted-n8-metal-sing-picorv32-nop-fetch.log
```

Expect 2 fetches, PASS $\sim$64 s/8. Use tiled BR; fused traps.

D.28 2-LUT cascade melt–emit (C48)

```
swift test -c release --filter testTwoLUTCascadeMeltFreezeSnapEmit
```

Eight Boolean corners of the snapped INIT must match $(a \wedge b) \oplus c$.

D.29 Boolean oracle benches (C1)

```
./Scripts/helut_boolean_bench.sh
./Scripts/helut_boolean_scale.sh
```

Bibliography

[1] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. TFHE: Fast fully homomorphic encryption over the torus. *Journal of Cryptology*, 33(1):34–91, 2020.

[2] Digital Defiance. Enigma 256 (E256) — polymorphic stream cipher, 2026. Enigma256.md.

[3] Digital Defiance. HELUT claim inventory, 2026. Living. Canonical source: directives/claim-sheet.md.

[4] Digital Defiance. HELUT: Homomorphic edge look-up tensors, 2026. Project report. Canonical source: paper/helut.tex.

[5] Digital Defiance. HELUT parameter cookbook, 2026. directives/parameter-cookbook.md.

[6] Digital Defiance. Metal torus compiler — phase 1 / phase 2, 2026. directives/metal-compiler-phases.md.

[7] Digital Defiance. Potential avenues, 2026. directives/potential-avenues.md.

[8] Digital Defiance. Research-release doctrine, 2026. directives/research-release.md.

[9] Digital Defiance. Research trajectory, 2026. directives/research-trajectory.md.

[10] Digital Defiance. Theorem (TensorLUT continuous-to-discrete), 2026. directives/tensorlut-theorem.md. Claim C19.

[11] Léo Ducas and Daniele Micciancio. FHEW: Bootstrapping homomorphic encryption in less than a second. In *EUROCRYPT*, 2015.

[12] Daniel D. Gajski, Samar Abdi, Andreas Gerstlauer, and Gunar Schirner. *Embedded System Design*. Springer, 2009.

[13] David Money Harris and Sarah L. Harris. *Digital Design and Computer Architecture*. Morgan Kaufmann, 2012.

[14] Vadim Lyubashevsky, Chris Peikert, and Oded Regev. On ideal lattices and learning with errors over rings. In *EUROCRYPT*, 2010.

[15] Christof Paar and Jan Pelzl. *Understanding Cryptography*. Springer, 2009.

[16] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. In *STOC*, 2005.

[17] Claire Wolf. PicoRV32 — a size-optimized RISC-V CPU. https://github.com/YosysHQ/picorv32.

[18] Claire Wolf. Yosys open SYnthesis suite. https://github.com/YosysHQ/yosys.